

DM_第三組

成人人口普查收入

指導教授：鮑永誠 老師

組長：111029208 吳文秀

組員：112029043 朱芯僊

112029012 翁鈺淇

112029030 魏麒恩

112029040 劉涵雲

目錄

壹、資料描述	
貳、資料前處理	
參、資料視覺化	
肆、問題假說	
伍、模型建立	
一、決策樹	
二、隨機森林	
三、羅吉斯迴歸	
陸、教授提問與問題回覆、解決策略	
柒、附錄	
一、工作分配表	
二、討論過程與組員照片	
捌、心得感想	
玖、參考資料	

壹、資料描述

(一)資料介紹

本資料來源由 RONNY KOHAVI 和 BARRY BECKER(數據挖掘和可視化，SILICON GRAPHICS)從 1994 年人口普查局數據庫中提取。統計目的是為了研究一個人年收入是否超過五萬美元，欄位包含年齡、性別、工作單位、教育程度、職業、種族、工作時數、收入等等，共有 32,561 筆資料，16 個資料欄位。

(二)探究動機

我們選擇這個主題，是因為「收入」可能與「教育、性別、年齡」等人口特徵因素密切相關，這些因素其實都反映了社會階層與資源分配的問題。透過成人人口普查資料，也許可以幫助政策制定者了解不同族群在經濟上的落差，進一步作為制定社會政策與教育政策的重要參考依據。

(三)原始資料說明

1. 連續型資料

<u>欄位名稱</u>	<u>說明</u>	<u>範例值</u>
<u>age</u>	<u>年齡（以歲為單位）</u>	<u>17 - 90</u>
<u>fnlwgt</u>	<u>最終權重，</u> <u>代表該筆樣本在母體</u> <u>中的代表性</u>	<u>12285 - 1,484,705</u>
<u>education.num</u>	<u>教育程度數值化</u> <u>（年級數）</u>	<u>1 - 16</u>

<u>capital.gain</u>	<u>資本收益</u>	<u>0 - 99, 999</u>
<u>capital.loss</u>	<u>資本損失</u>	<u>0 - 4, 356</u>
<u>hours.per.week</u>	<u>每週工作時數</u>	<u>1 - 99</u>

2. 類別型資料

<u>欄位名稱</u>	<u>說明</u>	<u>範例值</u>
<u>workclass</u>	<u>工作類別（如政府、私人企業、自營等）</u>	<u>Private, Self-emp-not-inc</u>
<u>education</u>	<u>教育程度名稱</u>	<u>HS-grad, Bachelors</u>
<u>marital.status</u>	<u>婚姻狀況</u>	<u>Married-civ-spouse, Divorced, Widowed, Separated, Never-married</u>
<u>occupation</u>	<u>職業類別</u>	<u>Prof-specialty, Sales</u>
<u>relationship</u>	<u>家庭關係</u>	<u>Husband, Not-in-family</u>
<u>race</u>	<u>種族</u>	<u>White, Black, Asian-Pac-Islander</u>
<u>sex</u>	<u>性別</u>	<u>Male, Female</u>
<u>native.country</u>	<u>原生國家</u>	<u>United-States, Mexico</u>
<u>income</u>	<u>收入分類（是否超過 50,000 美元）</u>	<u><=50K, >50K</u>

貳、資料前處理

(一)將資料編碼成有意義的單位

我們在初次執行後續模型時，因準確度極低，故在課堂小組討論期間向老師提問，老師建議我們可以把資料定義得更清楚，否則跑出來的結果意義不大且難以解釋。

1. age_cat
 - a. 18~25 歲設成 1，為青少年
 - b. 26~35 歲設成 2，為青壯年
 - c. 36~65 歲設成 3，為壯年
2. wor_cat
 - a. "0" 為無薪或其他工作
 - b. "1" 為私人或自營企業
 - c. "2" 為政府相關企業
3. edun_cat
 - a. "1" 為 0-5 級 表示"教育程度低"
 - b. "2" 為 6-10 級 表示"教育程度中"
 - c. "3" 為 11-15 級 表示"教育程度中高"
 - d. "4" 為 16-20 級 表示"教育程度高"
4. mar_cat
 - a. "0" 表示未婚或非伴侶
 - b. "1" 表示已婚
 - c. "2" 表示其他(分居/喪偶)
5. occ_cat
 - a. "0" 為低技能
 - b. "1" 為技術中等
 - c. "2" 為高技能
6. rel_cat
 - a. "0" 為無家庭關係
 - b. "1" 為旁系
 - c. "2" 為直系
7. sex_cat
 - a. 男性設為 1
 - b. 女性設為 2
8. income_bin
 - a. $\leq 50k$ 美元設成 0，為低收入
 - b. $> 50k$ 美元設成 1，為高收入

*income 在原始資料中就只有「 $\leq 50k$ 」、「 $> 50k$ 」兩種分

9. net_cat

- "0" 為發展落後地區
- "1" 為新興經濟體
- "2" 為已開發國家

10. race_cat

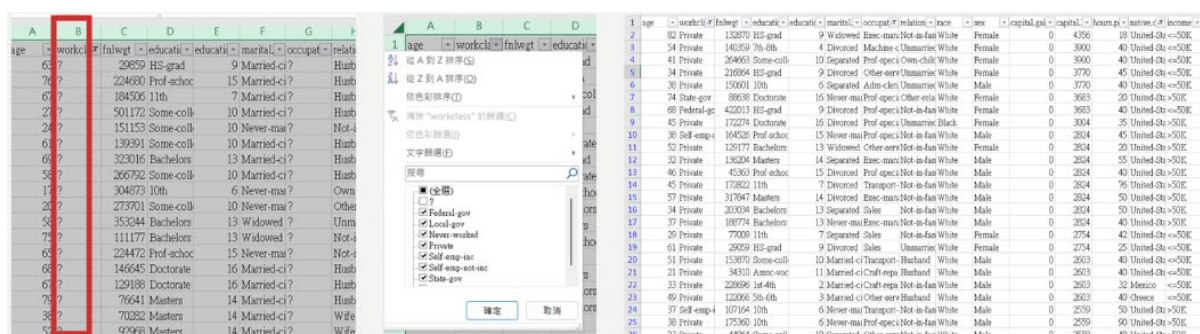
- "1" 為白人
- "2" 為黑人
- "3" 為亞裔美國人
- "4" 為愛斯基摩人
- "5" 為其他

11. edu_cat

- "0" 為低學歷(國小以下)
- "1" 為中學歷(國高中)
- "2" 為高學歷(大學以上)

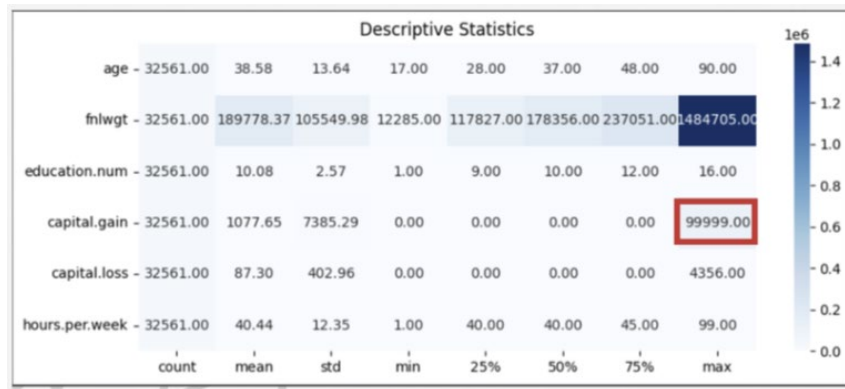
(二)處理遺漏值及異常值

我們使用 Excel 將資料全部選取後點選資料點擊篩選問號找出遺漏值，並將其刪除。如(三)的檢查表所示，有遺漏值的數據有 workclass、occupation、native.country，我們將有遺漏值的欄位通通以相同方式處理乾淨。



圖一、刪除遺漏值

我們首先透過 Google Colab 進行資料檢視，發現資料中存在明顯偏離之異常值「99999.00」。因此，處理方式比照遺漏值刪除流程，使用 Excel 將資料全數選取後，透過「資料 → 篩選」功能篩選出數值為 99999 的紀錄，並將其視為異常值予以刪除。如(三)之檢查表所示，含有遺漏值與異常值的欄位僅出現在 capital.gain (資本收益) 變項中，故本研究針對該欄位之異常資料，統一採用相同方式進行清理，以確保資料品質與一致性。



圖二、異常值發現

此外，由於原始資料中年齡分布範圍為 10 至 99 歲，為符合研究目的並依據相關文獻之人口年齡分類，本研究將分析對象限定為 18 至 65 歲。考量 18 歲以下多不具備完整勞動參與能力，而 65 歲以上多已進入退休階段，與本研究之收入分析情境不符，因此將不在此年齡區間內之個案整列視為異常資料並予以刪除。

(四)google colab 檢查

將資料整理後，使用 google colab 檢查可以得知各項資料有無遺漏值，以及遺漏值總共有多少。



圖三 遺漏值檢查

(五)資料平衡

1. 整理完資料後，由於我們主要想分析「收入與其他特徵的關係(什麼特徵影響收入最重大)」，我們發現在原始資料中我們的資料是不平衡且差距極大的，因此小組決定針對低收做下採樣而針對高收做 SMOTE 上採樣的

方式來讓資料平衡。

如附圖所示:平衡後 income=0 以及 income=1 皆為 11540 筆。

```
===== 原始資料總覽 =====
原始資料量: (28852, 26)
原始收入分布:
income_bin
0.0    21676
1.0     7176
Name: count, dtype: int64

訓練集大小: (23081, 13)
訓練集收入分布:
income_bin
0.0    17340
1.0     5741
Name: count, dtype: int64
測試集大小: (5771, 13)
測試集收入分布:
income_bin
0.0     4336
1.0     1435
Name: count, dtype: int64

===== 平衡後訓練集總覽 =====
平衡後訓練集大小: (23080, 13)
平衡後訓練集分布:
income_bin
0.0    11540
1.0    11540
Name: count, dtype: int64
```

2. 將資料平衡至一樣的數目後，我們需要處理剩下的負樣本。
3. 以下為整理好的資料，資料筆數由原本的 32561 筆 ➡ 28852 筆。

統計量

income_bin

N	有效	28852
	遺漏	0
平均數		.2487
中位數		.0000

income_bin

		次數分配表	百分比	有效百分比	累積百分比
有效	<=50K	21676	75.1	75.1	75.1
	>50K	7176	24.9	24.9	100.0
	總計	28852	100.0	100.0	

參、資料視覺化

(一)熱力圖

熱力圖程式碼截圖

```
# Step 1. 上傳 .sav 檔案
from google.colab import files
upload = files.upload()

# Step 2. 安裝並載入套件
!pip install pyreadstat

import pyreadstat
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Step 3. 讀取 .sav 檔案
file_name = list(upload.keys())[0]
df, meta = pyreadstat.read_sav(file_name)

print("資料筆數與欄位形狀:", df.shape)
display(df.head())

# Step 4. 選取原始數值型欄位 (適合相關係數)
numeric_cols = [
    'age',
    'fnlwt',
    'education.num',
    'capital.gain',
    'capital.loss',
    'hours.per.week',
    'income_bin'
]

print("可用於相關係數的欄位:")
print(numeric_cols)

# Step 5. 計算相關矩陣
corr = df[numeric_cols].corr()
display(corr)

# Step 6. 繪製熱力圖
plt.figure(figsize=(12,10))
sns.heatmap(corr, annot=True, cmap='coolwarm', fmt=".2f")
plt.title("Correlation Heatmap (Numerical Variables)")
plt.show()
```

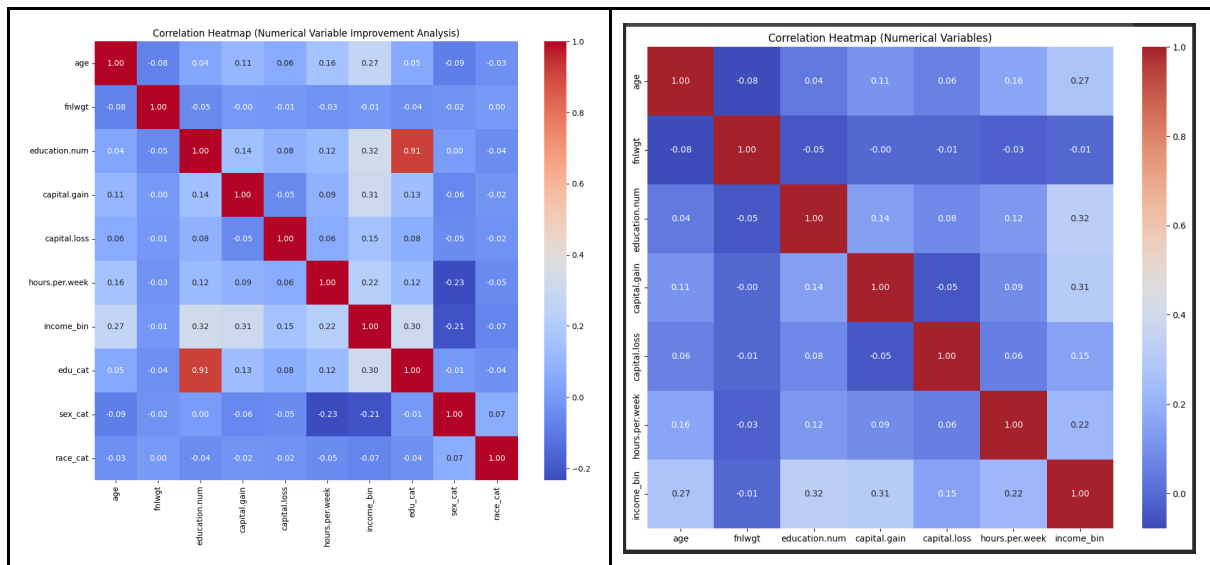
我們使用 Python 繪製相關係數熱力圖，以前處理完成後之資料作為分析基礎，以視覺化方式呈現各數值型變數之間的線性相關程度。

由熱力圖結果可觀察到，收入（income_bin）與教育年數（education.num）及教育類別（edu_cat）分別呈現中度正相關（約 0.37 與 0.34），顯示教育程度越高，獲得較高收入之可能性亦隨之提升；此外，收入與每週工作時數（hours.per.week）亦呈現正相關（約 0.26），顯示工作投入程度可能與收入水準具有一定關聯。

相較之下，收入與性別變數呈現負相關，顯示性別差異仍可能影響收入分布。基於上述結果，本研究進一步透過視覺化分析，分別探討教育程度搭配性別，以及工作時數搭配職業類別下之收入分布情形，以深化對相關變項與收入關係之理解，並作為後續統計分析之依據。

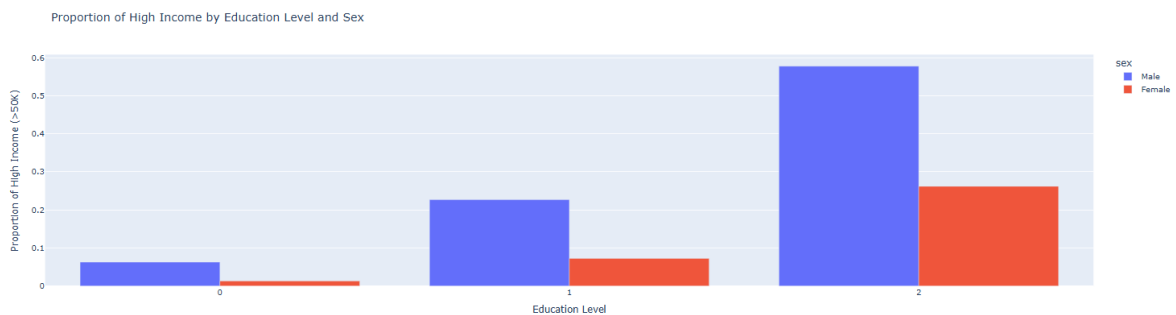
含編碼後類別型資料熱力圖

只有連續型資料與收入比較熱力圖



(二)其他視覺化圖表

1. 教育程度與性別收入

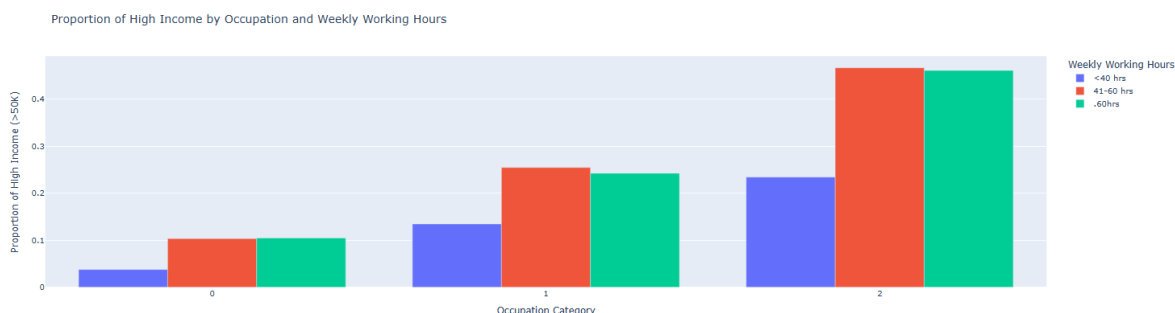


圖一呈現不同教育程度下，依性別區分之高收入（>50K）比例分布情形。

由圖中結果可觀察到，隨著教育程度提升，無論男性或女性，高收入比例皆呈現明顯上升趨勢，顯示教育程度與收入之間存在正向關聯。在低教育程度組別中，高收入比例普遍偏低；而在高教育程度組別中，高收入比例顯著提高，顯示教育年數對收入水準具有重要影響。

此外，在各教育層級中，男性的高收入比例皆高於女性，顯示即便在相同教育背景下，性別仍可能對收入結果產生影響。此結果與前述熱力圖中，收入與教育變數呈現中度正相關、且與性別呈現負相關之結果相互呼應，進一步支持教育程度與性別為影響收入的重要人口學因素。

2. 不同工作時長下的職業收入狀況



圖二呈現不同職業類別下，依每週工作時數區分之高收入比例分布情形。

結果顯示，在各職業類別中，每週工作時數較長者，其高收入比例普遍高於工時較短者，特別是在中、高職業類別中，此差異更為明顯。顯示工作投入程度（以每週工時衡量）可能與收入水準呈現正向關聯。

此外，在高職業類別中，當每週工時達 41 - 60 小時或超過 60 小時時，高收入比例顯著高於 40 小時以下族群，顯示工作時數與職業類別可能具有交互影響效果。此視覺化結果亦與相關係數分析中，收入與每週工作時數呈現正相關之結果一致，支持後續進一步納入工時變項進行多變量分析的合理性。

肆、問題假說

如前述分析所示，我們對資料集中之「income」變數特別關注。本研究以成人人口普查資料為基礎，建構高收入預測模型，並進一步分析影響高收入的關鍵人口與社經特徵。

Q1: 成人人口普查資料中，哪些人口與社經特徵能有效預測個體是否屬於高收入族群？

伍、模型建立

（一）決策樹

1. 模型介紹

決策樹是一種以樹狀結構進行分類與預測的模型，透過逐層判斷特徵條件，將資料分割成不同群組，最終得到預測結果。其結構主要可分為以下幾個部分：(1)根節點：表示所有數據的起始點；(2)內部節點：根據某一特徵作為判斷條件，將資料進行分割；(3)分支：依據內部節點的判斷結果所產生的資料分流方向；(4)葉節點：終極預測結果的節點，通常對應最終的類別。

1.1 決策樹的優與缺

其優點在於模型結構清楚，對於非技術背景的人也容易解釋與溝通。此外，決策樹對缺失值較不敏感，也不需要對資料進行標準化，能同時處理類別型與數值型變數，並可自然捕捉特徵之間的非線性關係，在實務應用上相當方便；缺點是容易對訓練資料過度擬合，特別是在樹的深度過深或樣本數較少時，導致模型在測試資料上的泛化能力下降。其次，單一決策樹對資料的微小變動較為敏感，在類別分布不平衡的資料集中，模型表現可能較不理想，造成穩定性不足，而相較於集成式方法（如隨機森林），單棵決策樹在整體預測效能上通常較為有限。

1.2 資訊量評估的方法：

1.2.1 熵 (Entropy)

作為衡量資料不確定性或混亂程度的指標。熵值越高，表示資料分布越混亂，模型進行分類的難度也越高；反之，熵值越低，代表資料越集中，模型越容易進行判斷與預測。

$$\text{Entropy}(S) = - \sum_{i=1}^m p_i \log_2 p_i$$

1.2.2 gini 係數

gini 係數用於衡量資料中各類別的純度，常作為決策樹進行節點分割時的評估指標。gini 係數值越低，代表該節點中的資料越集中於單一類別，純度越高。因此，在建立決策樹時，模型會選擇使 gini 不純度最小的特徵作為分割依據，以提升分類效果。

$$\text{Gini Impurity}(S) = 1 - \sum_{i=1}^k p_i^2$$

2. 模擬程式碼

```
# =====
# 1 套件
# =====
!pip install -q pyreadstat shap
import pandas as pd
import numpy as np
import pyreadstat
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import (
    confusion_matrix, accuracy_score,
    roc_curve, roc_auc_score,
    classification_report
)

# =====
# 2 讀取資料
# =====
train_df, _ = pyreadstat.read_sav("train_balanced.sav")
test_df, _ = pyreadstat.read_sav("test.sav")

feature_cols = [
    "work_cat", "edu_cat", "mar_cat", "occ_cat", "rel_cat",
    "race_cat", "sex_cat", "nat_cat", "educ_cat", "age_cat",
    "capital.gain", "capital.loss", "hours.per.week"
]

X_train = train_df[feature_cols]
y_train = train_df["income_bin"]

X_test = test_df[feature_cols]
y_test = test_df["income_bin"]

# =====
# 3 模型 & CV
# =====
model = DecisionTreeClassifier(
    criterion="gini",
    max_depth=4,
    min_samples_leaf=20,
    random_state=42
)

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# =====
# 4 儲存容器 (五折)
# =====
cm_train_list, cm_test_list = [], []
acc_train_list, acc_test_list = [], []

roc_train_list, roc_test_list = [], []
auc_train_list, auc_test_list = [], []

importance_list = []
report_list_train, report_list_test = [], []

# =====
# 5 五折交叉驗證 (每折: Train/Test CM + Train/Test ROC)
# =====
for fold, (train_idx, val_idx) in enumerate(cv.split(X_train, y_train), start=1):
    X_tr, X_val = X_train.iloc[train_idx], X_train.iloc[val_idx]
    y_tr, y_val = y_train.iloc[train_idx], y_train.iloc[val_idx]

    model.fit(X_tr, y_tr)

    # ===== 預測: Train / Test
    y_tr_pred = model.predict(X_tr)
    y_test_pred = model.predict(X_val)

    # ===== Train/Test accuracy (存起來)
    acc_train = accuracy_score(y_tr, y_tr_pred)
    acc_test = accuracy_score(y_val, y_test_pred)
    acc_train_list.append(acc_train)
    acc_test_list.append(acc_test)

    # ===== (A) Confusion Matrix (每折: Train + Test 同一張)
    cm_tr = confusion_matrix(y_tr, y_tr_pred)
    cm_te = confusion_matrix(y_val, y_test_pred)

    cm_train_list.append(cm_tr)
    cm_test_list.append(cm_te)

    plt.figure(figsize=(10, 4))

    plt.subplot(1, 2, 1)
    sns.heatmap(cm_tr, annot=True, fmt="d", cmap="Oranges")
    plt.title(f"Fold {fold} Train CM")
    plt.xlabel("Predicted")
    plt.ylabel("Actual")

    plt.subplot(1, 2, 2)
    sns.heatmap(cm_te, annot=True, fmt="d", cmap="Blues")
    plt.title(f"Fold {fold} Test CM")
    plt.xlabel("Predicted")
    plt.ylabel("Actual")

    plt.tight_layout()
    plt.show()

    # ===== (B) ROC (每折: Train + Test 同一張)
    y_tr_prob = model.predict_proba(X_tr)[:, 1]
    y_test_prob = model.predict_proba(X_val)[:, 1]

    fpr_tr, tpr_tr, _ = roc_curve(y_tr, y_tr_prob)
    fpr_te, tpr_te, _ = roc_curve(y_val, y_test_prob)

    roc_train_list.append((fpr_tr, tpr_tr))
    roc_test_list.append((fpr_te, tpr_te))

    auc_tr = roc_auc_score(y_tr, y_tr_prob)
    auc_te = roc_auc_score(y_val, y_test_prob)
    auc_train_list.append(auc_tr)
    auc_test_list.append(auc_te)

    plt.figure(figsize=(7, 6))
    plt.plot(fpr_tr, tpr_tr, label=f"Train AUC={auc_tr:.3f}")
    plt.plot(fpr_te, tpr_te, label=f"Test AUC={auc_te:.3f}")
    plt.plot([0, 1], [0, 1], '-', color='gray')
    plt.title(f"Fold {fold} ROC (Train vs Test)")
    plt.xlabel("FPR")
    plt.ylabel("TPR")
    plt.legend()
    plt.tight_layout()
    plt.show()

    # ===== 特徵重要性 (折內存起來做平均 ± std)
    importance_list.append(model.feature_importances_)

# ===== 分類報告 (Train 用 X_tr, Test 用同一份 X_test)
report_list_train.append(classification_report(y_tr, y_tr_pred, output_dict=True))
report_list_test.append(classification_report(y_val, y_test_pred, output_dict=True))
```

```
# =====
# 6 平均混淆矩陣 (Train / Test)
# =====
cm_train_mean = np.mean(np.array(cm_train_list), axis=0)
cm_test_mean = np.mean(np.array(cm_test_list), axis=0)
```

```
plt.figure(figsize=(10,4))
```

```
plt.subplot(1,2,1)
sns.heatmap(cm_train_mean, annot=True, fmt=".1f", cmap="Oranges")
plt.title("Mean Train Confusion Matrix (5-Fold)")
plt.xlabel("Predicted")
plt.ylabel("Actual")
```

```
plt.subplot(1,2,2)
sns.heatmap(cm_test_mean, annot=True, fmt=".1f", cmap="Blues")
plt.title("Mean Test Confusion Matrix (5-Fold)")
plt.xlabel("Predicted")
plt.ylabel("Actual")
```

```
plt.tight_layout()
plt.show()
```

```
# =====
# 7 Mean ROC (Train + Test 同一張, 含 AUC 平均±標準差)
# =====
mean_fpr = np.linspace(0, 1, 200)
```

```
tpr_tr_interp = np.array([np.interp(mean_fpr, fpr, tpr) for fpr, tpr in roc_train_list])
tpr_te_interp = np.array([np.interp(mean_fpr, fpr, tpr) for fpr, tpr in roc_test_list])
```

```
mean_tpr_tr = tpr_tr_interp.mean(axis=0)
std_tpr_tr = tpr_tr_interp.std(axis=0)
```

```
mean_tpr_te = tpr_te_interp.mean(axis=0)
std_tpr_te = tpr_te_interp.std(axis=0)
```

```
mean_auc_tr = np.mean(auc_train_list)
std_auc_tr = np.std(auc_train_list)
```

```
mean_auc_te = np.mean(auc_test_list)
std_auc_te = np.std(auc_test_list)
```

```
plt.figure(figsize=(8,6))
plt.plot(mean_fpr, mean_tpr_tr, label=f"Mean Train ROC (AUC={mean_auc_tr:.3f} ± {std_auc_tr:.3f})")
plt.fill_between(mean_fpr, mean_tpr_tr-std_tpr_tr, mean_tpr_tr+std_tpr_tr, alpha=0.2)
```

```
plt.plot(mean_fpr, mean_tpr_te, label=f"Mean Test ROC (AUC={mean_auc_te:.3f} ± {std_auc_te:.3f})")
plt.fill_between(mean_fpr, mean_tpr_te-std_tpr_te, mean_tpr_te+std_tpr_te, alpha=0.2)
```

```
plt.plot([0,1],[0,1], '--', color='gray')
plt.xlabel("FPR")
plt.ylabel("TPR")
```

```
plt.title("Mean ROC over 5 Folds (Train vs Test)")
plt.legend()
plt.tight_layout()
plt.show()
```

```
# =====
# 8 平均特徵重要性 (五折平均 ± 標準差) + 圖
# =====
```

```
importance_arr = np.array(importance_list)
feat_df = pd.DataFrame({
    "feature": feature_cols,
    "mean_importance": importance_arr.mean(axis=0),
    "std_importance": importance_arr.std(axis=0)
}).sort_values("mean_importance", ascending=True)
```

```
plt.figure(figsize=(10,6))
```

```
plt.barh(
    y=feat_df["feature"],
    width=feat_df["mean_importance"],
    xerr=feat_df["std_importance"].to_numpy(),
    capsize=5
)
```

```
plt.xlabel("Mean Feature Importance ± Std")
plt.ylabel("Feature")
plt.title("Feature Importance over 5 folds")
plt.tight_layout()
plt.show()
```

```
# =====
# 9 五折分類報告 (mean / std / mean±std) — 取 positive 類
# =====
```

```
def five_fold_classification_stats(reports):
    metrics = ["precision", "recall", "f1-score"]
    sample_keys = reports[0].keys()
    class_keys = [k for k in sample_keys if k not in ["accuracy", "macro avg", "weighted avg"]]
    positive_key = sorted(class_keys)[-1] # 預設取較大的那類當 positive
```

```
mean_dict, std_dict, mean_std_dict = [], [], []
```

```
for m in metrics:
    values = np.array([r[positive_key][m] for r in reports])
    mean_dict[m] = values.mean()
    std_dict[m] = values.std()
    mean_std_dict[m] = f"{values.mean():.6f} ± {values.std():.6f}"
```

```
support = np.mean([r[positive_key]["support"] for r in reports])
```

```
mean_df = pd.DataFrame(mean_dict, index=["positive"])
std_df = pd.DataFrame(std_dict, index=["positive"])
mean_std_df = pd.DataFrame(mean_std_dict, index=["positive"])
```

```
mean_df["support"] = support
std_df["support"] = support
mean_std_df["support"] = support
return mean_df, std_df, mean_std_df
```

```
mean_report, std_report, mean_std_report = five_fold_classification_stats(report_list_test)
```

```
# =====
# 10 平均分類報告 (Train / Test)
# =====
```

```
def average_classification_report(reports):
    keys = list(reports[0].keys())
    avg_report = {}
    for k in keys:
        if k not in ["accuracy", "macro avg", "weighted avg"]:
            avg_report[k] = {
                "precision": np.mean([r[k]["precision"] for r in reports]),
                "recall": np.mean([r[k]["recall"] for r in reports]),
                "f1-score": np.mean([r[k]["f1-score"] for r in reports]),
                "support": np.mean([r[k]["support"] for r in reports]),
            }
    return avg_report
```

```
avg_report_train = average_classification_report(report_list_train)
avg_report_test = average_classification_report(report_list_test)
```

```
# 11 訓練整個訓練集並畫決策樹
```

```
# =====
model.fit(X_train, y_train)
plt.figure(figsize=(20,12))
plot_tree(
    model,
    feature_names=feature_cols,
    class_names=["False", "True"],
    filled=True,
    rounded=True,
    fontsize=10
)
```

```
plt.tight_layout()
plt.show()
```

```
# =====
# 12 SHAP 特徵重要性分析 (bar + beeswarm)
# =====
```

```
import shap
model.fit(X_train, y_train)
```

```
sample_size = min(2000, len(X_test))
X_shap = X_test.sample(n=sample_size, random_state=42)
```

```
explainer = shap.Explainer(model, X_train)
sv = explainer(X_shap)
```

```
vals = sv.values
shap_pos = vals[:, :, 1] if vals.ndim == 3 else vals
```

```
mean_abs_shap = np.abs(shap_pos).mean(axis=0)
shap_importance_df = pd.DataFrame({
    "feature": X_shap.columns,
    "mean_abs_shap": mean_abs_shap
}).sort_values("mean_abs_shap", ascending=False)
```

```
plt.figure(figsize=(10,6))
shap.summary_plot(shap_pos, X_shap, plot_type="bar", show=False)
plt.title("SHAP Feature Importance (bar)")
plt.show()
```

```
plt.figure(figsize=(10,6))
shap.summary_plot(shap_pos, X_shap, show=False)
plt.title("SHAP Summary (beeswarm)")
plt.show()
```

```
# =====
# 1.2 文字輸出
# =====
acc_train_arr = np.array(acc_train_list)
acc_test_arr = np.array(acc_test_list)

mean_train_acc = acc_train_arr.mean()
std_train_acc = acc_train_arr.std()

mean_test_acc = acc_test_arr.mean()
std_test_acc = acc_test_arr.std()

def _df_to_block(df, round_n=6):
    """把 DataFrame 轉成像截圖那樣的純文字區塊"""
    return df.round(round_n).to_string()

print(f"Train Accuracy: Mean={mean_train_acc:.4f}, Std={std_train_acc:.4f}")
print(f"Test Accuracy: Mean={mean_test_acc:.4f}, Std={std_test_acc:.4f}\n")

print("==== Average Classification Report - Train ====")
print(_df_to_block(pd.DataFrame(avg_report_train).T))
print()

print("==== Average Classification Report - Test ====")
print(_df_to_block(pd.DataFrame(avg_report_test).T))
print()

print("==== 五折交叉驗證準確率 ====")
for i, acc in enumerate(acc_test_arr, 1):
    print(f"Fold {i}: {acc:.6f}")
print()

print("==== 平均準確率 ====")
print(f"Mean Accuracy: {mean_test_acc:.6f}")
print(f"Std Accuracy : {std_test_acc:.6f}")
print()
```

```
print("==== 五折偏差 (每折 - 平均) ====")
for i, acc in enumerate(acc_test_arr, 1):
    print(f"Fold {i}: {acc - mean_test_acc:+.6f}")
print()

print("==== 五折交叉驗證「平均分類報告」 ====")
print(_df_to_block(mean_report))
print()

print("==== 五折交叉驗證「標準差分類報告」 ====")
print(_df_to_block(std_report))
print()

print("==== 五折交叉驗證「平均 ± 標準差 分類報告」 ====")
print(_df_to_block(mean_std_report))
print()

print("==== Mean Feature Importance ====")
print(_df_to_block(feats_df, round_n=6))
print()

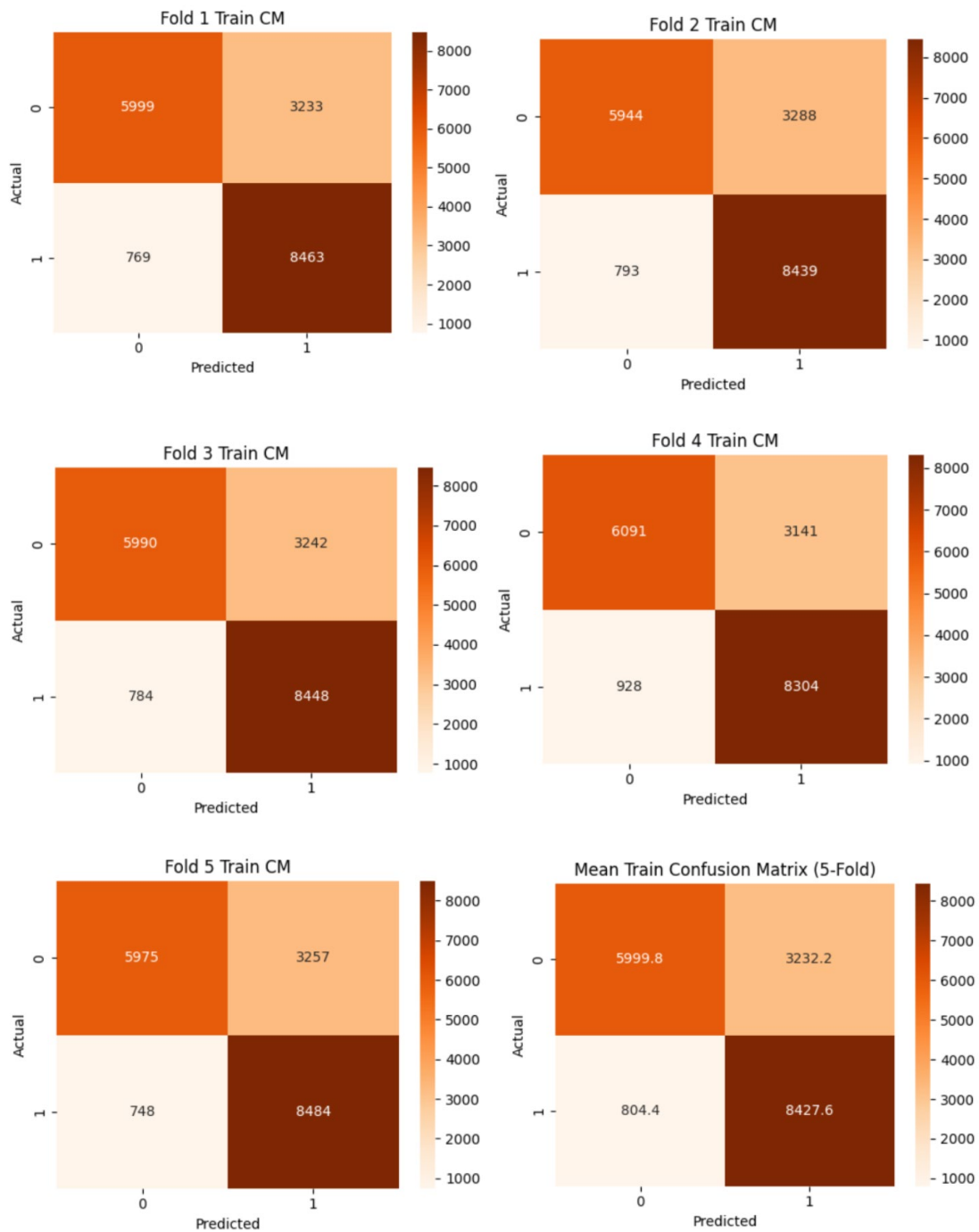
print("==== SHAP Feature Importance (mean |SHAP|) ====")
print(_df_to_block(shap_importance_df, round_n=6))
```

3. 決策樹超參數

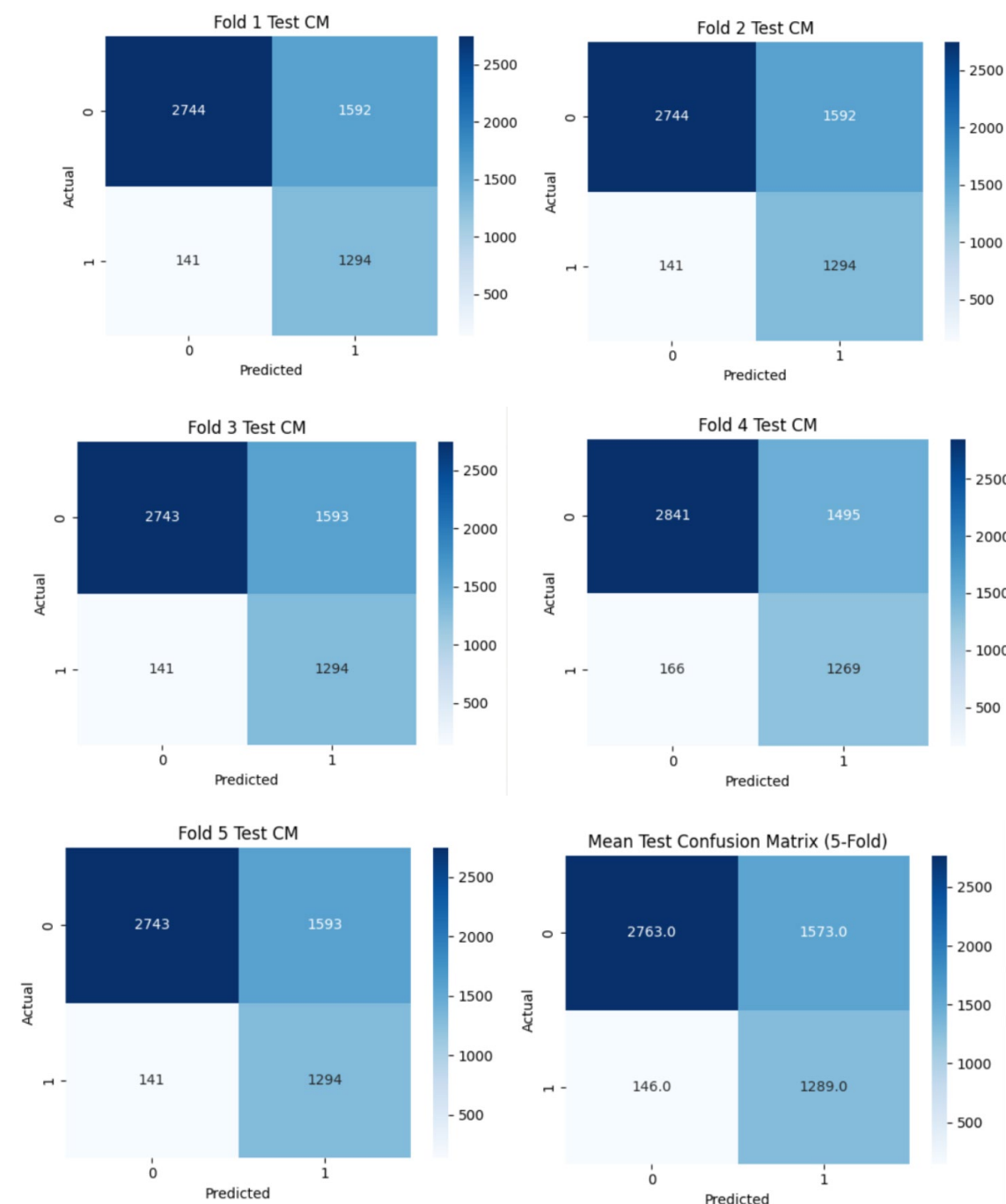
抽資料方式	80/20 法則
最大樹深	4
樹葉節點 (每一個分支的最少樣本數)	20
亂數種子	42

4. 利用混淆矩陣檢驗模型

4.1 訓練集：



4.2 測試集:



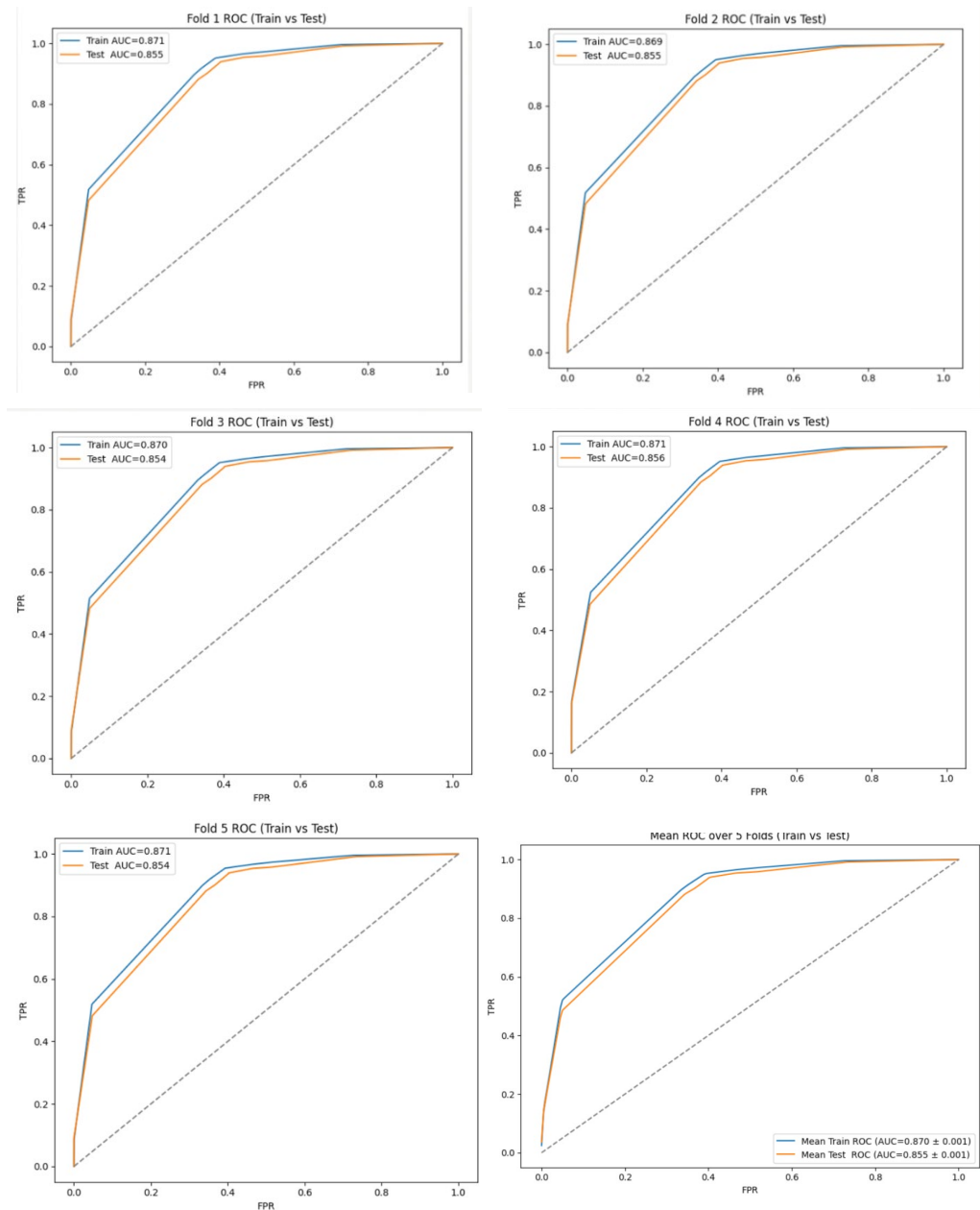
5. ROC 曲線

本研究透過五折交叉驗證評估決策樹模型於訓練集與測試集上的分類效能，並以 ROC 曲線及 AUC 作為主要判別能力指標。由各折 ROC 曲線結果可觀察到，訓練集 AUC 約介於 0.869 至 0.871 之間，而測試集 AUC 則約落在 0.854 至 0.856 之間，顯示模型在不同資料切分下皆能維持穩

定的分類表現，未出現明顯過度擬合的情形。進一步彙整五折結果後，模型的平均訓練集 AUC 約為 0.870，平均測試集 AUC 約為 0.855，且標準差極小，顯示模型具有良好的一致性與穩定性。依據 AUC 指標之模型判別能力分級標準表(5.1)來看，AUC 落於 0.8 至 0.9 之「良好」區間，代表本研究所建構之決策樹模型具有一定的分辨能力。

AUC 數值	模型程度
< 0.7	待改進
0.7 ~ 0.8	一般
0.8 ~ 0.9	良好
0.9 以上	非常良好

5.1 AUC 指標之模型判別能力分級表



5.2 ROC 曲線圖

6. 特徵重要性:

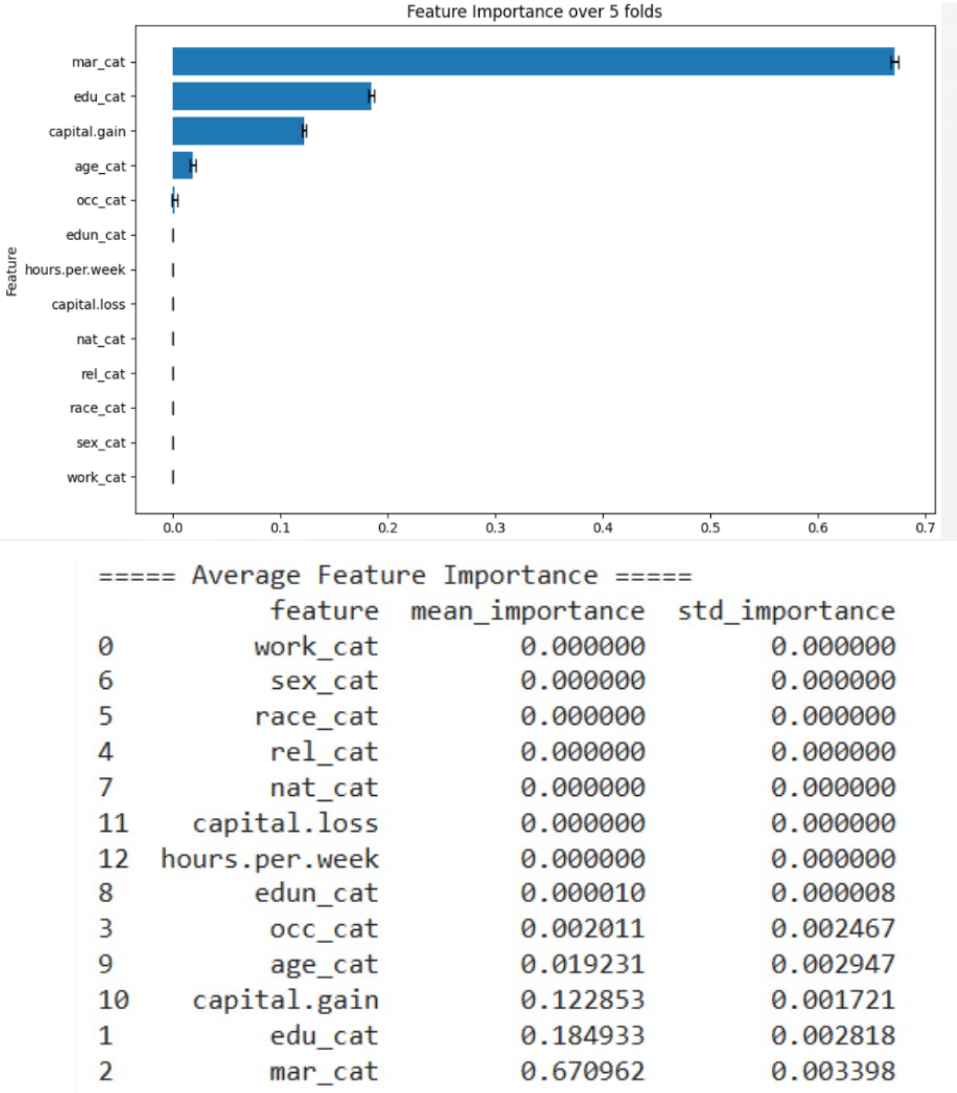
6.1 特徵重要性分析

由於我們主要關注各項特徵對「income」變數的重要程度，因此在建立決策樹模型之前，先進行特徵重要性分析，以了解不同特徵對收入預測的影響程度，並作為後續決策樹建模的參考。特徵重要性係根據各特徵於樹結構中所貢獻的不純度下降 (Gini impurity decrease) 進行

計算，並彙整五折交叉驗證結果，取其平均值與標準差，以評估特徵影響之穩定性。

由圖 6.1.1 結果可知，婚姻狀態（mar_cat）為影響模型判斷最關鍵之特徵，其平均重要性約為 0.67，顯著高於其他變數，顯示該特徵在模型分裂節點中扮演主要決策角色。其次為教育程度（edu_cat）與資本收益（capital.gain），其平均重要性分別約為 0.18 與 0.12，顯示其對收入分類亦具有一定影響。

相較之下，其餘特徵如年齡類別（age_cat）、職業類別（occ_cat）及教育年數類別（edun_cat）之重要性皆明顯較低，而部分變數（如性別、種族、工作類型等）在本模型中之重要性接近於零，表示其在決策樹分裂過程中未被頻繁選用，對最終分類結果影響有限。

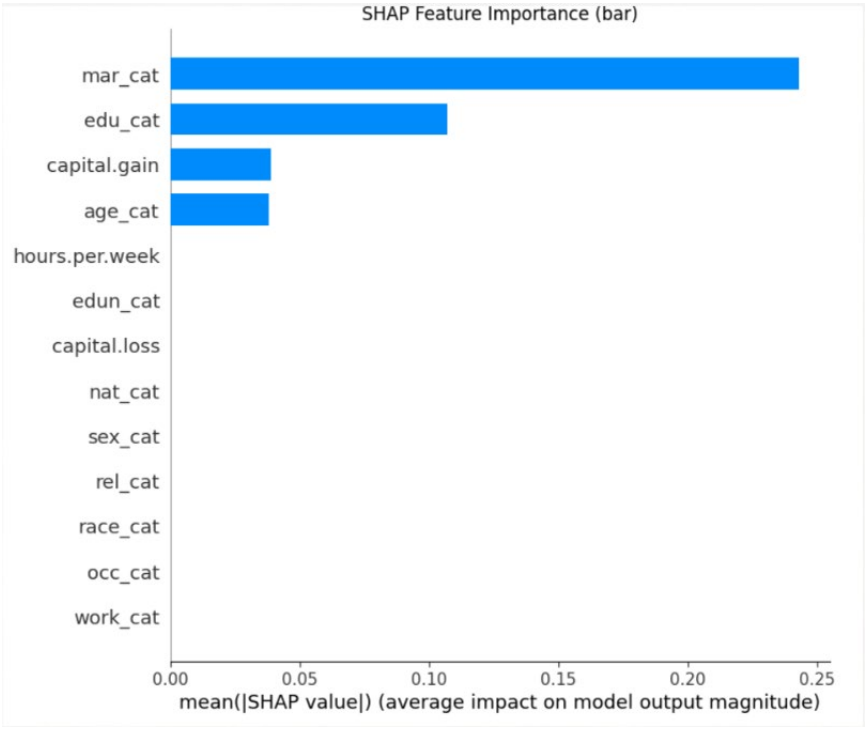


6.1.1 特徵重要性比較圖

6.2 SHAP 特徵貢獻分析

為進一步提升模型決策過程之可解釋性，本研究採用 SHAP (SHapley Additive exPlanations) 方法，分析各特徵對決策樹模型輸出結果的實際貢獻程度。SHAP 值源自合作賽局理論，能夠量化單一特徵在不同樣本中對模型預測結果的正向或負向影響，並同時兼顧全域與區域層級的解釋能力。

由 SHAP 全域特徵重要性(圖 6.2.1)可知，婚姻狀態 (mar_cat) 為影響模型輸出的最關鍵特徵，其平均 SHAP 絕對值顯著高於其他變數，顯示該特徵在整體預測過程中對模型判斷具有最大影響力。其次為教育程度 (edu_cat)，而資本收益 (capital.gain) 與年齡類別 (age_cat) 亦對模型預測具有一定程度的貢獻。其餘變數之平均 SHAP 值接近於零，表示其對模型最終輸出的影響相對有限。



===== SHAP Feature Importance (mean SHAP) =====		
	feature	mean_abs_shap
2	mar_cat	0.242884
1	edu_cat	0.107016
10	capital.gain	0.039008
9	age_cat	0.038123
0	work_cat	0.000000
4	rel_cat	0.000000
3	occ_cat	0.000000
5	race_cat	0.000000
6	sex_cat	0.000000
8	edun_cat	0.000000
7	nat_cat	0.000000
11	capital.loss	0.000000
12	hours.per.week	0.000000

圖 6.2.1 SHAP 全域特徵重要性

進一步觀察 SHAP 值分佈摘要圖（圖 6.2.2），可同時分析特徵數值大小與預測方向之關係。結果顯示，婚姻狀態與教育程度在不同樣本中對模型輸出具有明顯的正負向影響分布，顯示模型不僅依賴該些特徵進行判斷，且其影響方向會隨特徵取值而改變。資本收益則呈現較為明確的正向影響趨勢，高數值樣本通常對應較高的預測輸出，而年齡類別亦呈現類似但影響幅度較小的情形。

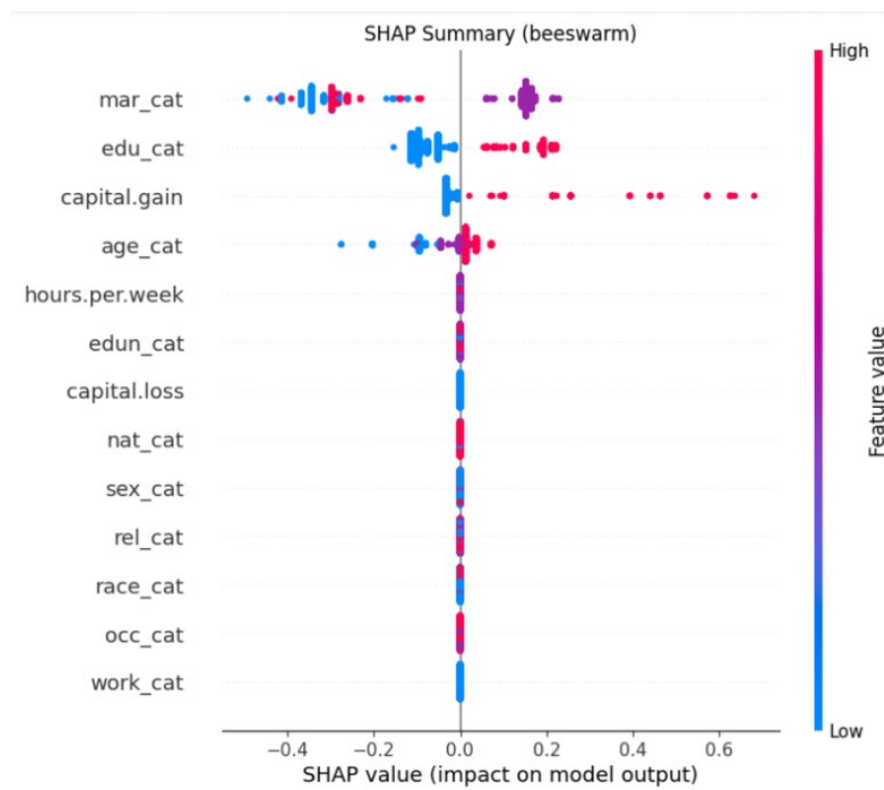


圖 6.2.2 SHAP 值分佈摘要圖

7. 交叉驗證與模型準確度

本研究透過五折交叉驗證評估決策樹模型之分類效能，由圖 7.1 可知，模型在訓練集上的平均準確率約為 0.781，標準差僅 0.0018，顯示模型在訓練資料上表現穩定；而測試集平均準確率約為 0.702，標準差為 0.0050，顯示模型於未見資料上仍具一定泛化能力，但整體表現略低於訓練集。進一步觀察平均分類報告結果，在測試集中，正類別（高收入）之 recall 值約為 0.898，顯示模型能有效辨識多數正類樣本；然而其 precision 約為 0.450，表示模型在預測為正類時，仍存在一定比例的誤判情形，導致整體 F1-score 約為 0.600。此結果反映模型偏向提高召回率，以犧牲部分精確率作為代價。而各折測試準確率分布相當集中，五折偏差幅度小，顯示模型在不同資料切分下之表現具備一致性與穩定性。整體而言，本模型能穩定區分主要類別特徵，並在正類辨識上具有良好能力，但在精確率方面仍有進一步優化空間。

```

Train Accuracy: Mean=0.7814, Std=0.0018
Test Accuracy: Mean=0.7021, Std=0.0050

===== Average Classification Report - Train =====
      precision    recall  f1-score   support
0.0      0.881892    0.649892    0.748277     9232.0
1.0      0.722805    0.912868    0.806776     9232.0

===== Average Classification Report - Test =====
      precision    recall  f1-score   support
0.0      0.949854    0.637223    0.762695     4336.0
1.0      0.450458    0.898258    0.599979     1435.0

===== 五折交叉驗證準確率 =====
Fold 1: 0.699705
Fold 2: 0.699705
Fold 3: 0.699532
Fold 4: 0.712182
Fold 5: 0.699532

===== 平均準確率 =====
Mean Accuracy: 0.702131
Std Accuracy : 0.005026

===== 五折偏差 (每折 - 平均) =====
Fold 1: -0.002426
Fold 2: -0.002426
Fold 3: -0.002599
Fold 4: +0.010050
Fold 5: -0.002599

===== 五折交叉驗證「平均分類報告」 =====
      precision    recall  f1-score   support
positive    0.450458    0.898258    0.599979     1435.0

===== 五折交叉驗證「標準差分類報告」 =====
      precision    recall  f1-score   support
positive    0.00433    0.006969    0.002226     1435.0

===== 五折交叉驗證「平均 ± 標準差 分類報告」 =====
      precision          recall    f1-score   support
positive  0.450458 ± 0.004330  0.898258 ± 0.006969  0.599979 ± 0.002226     1435.0

```

圖 7.1 模型整體效能評估摘要

8. 最終決策樹：圖 8 為最終決策樹的結果。

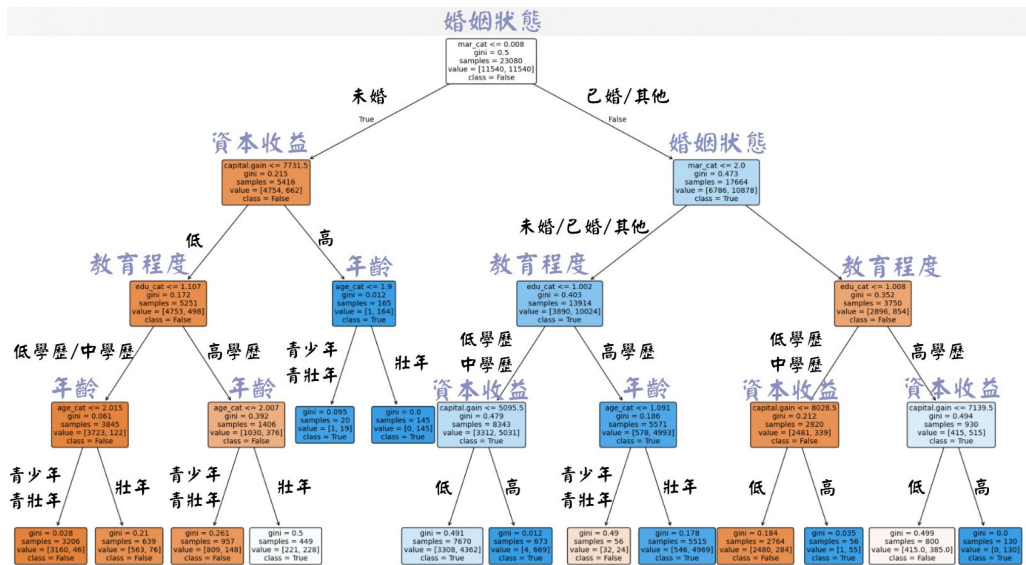


圖 8 完整圖

8.1 細節解釋及說明

8.1.1 根節點

本決策樹的根節點以婚姻狀態作為第一個分裂條件，顯示婚姻狀態是整體模型中最具判別力的特徵。在全體樣本中，模型首先將資料區分為「未婚」與「已婚／其他」，此分裂能有效降低 Gini 不純度，代表婚姻狀態對於預測收入高低具有關鍵影響。

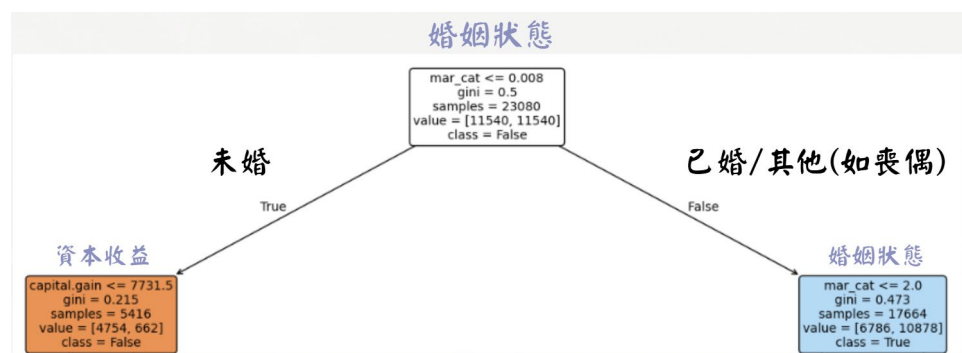


圖 8.1.1 根節點

8.1.2 左子樹

在未婚族群中，模型接著以資本收益進行分裂，顯示是否有顯著資本收益是影響收入的重要因素。當資本收益偏低時，模型

進一步參考教育程度與年齡進行判斷，整體結果多傾向於低收入類別；而在資本收益較高或年齡較大的情況下，則較容易被分類為高收入。此子樹反映未婚者的收入主要受個人資本與人力資本條件影響。

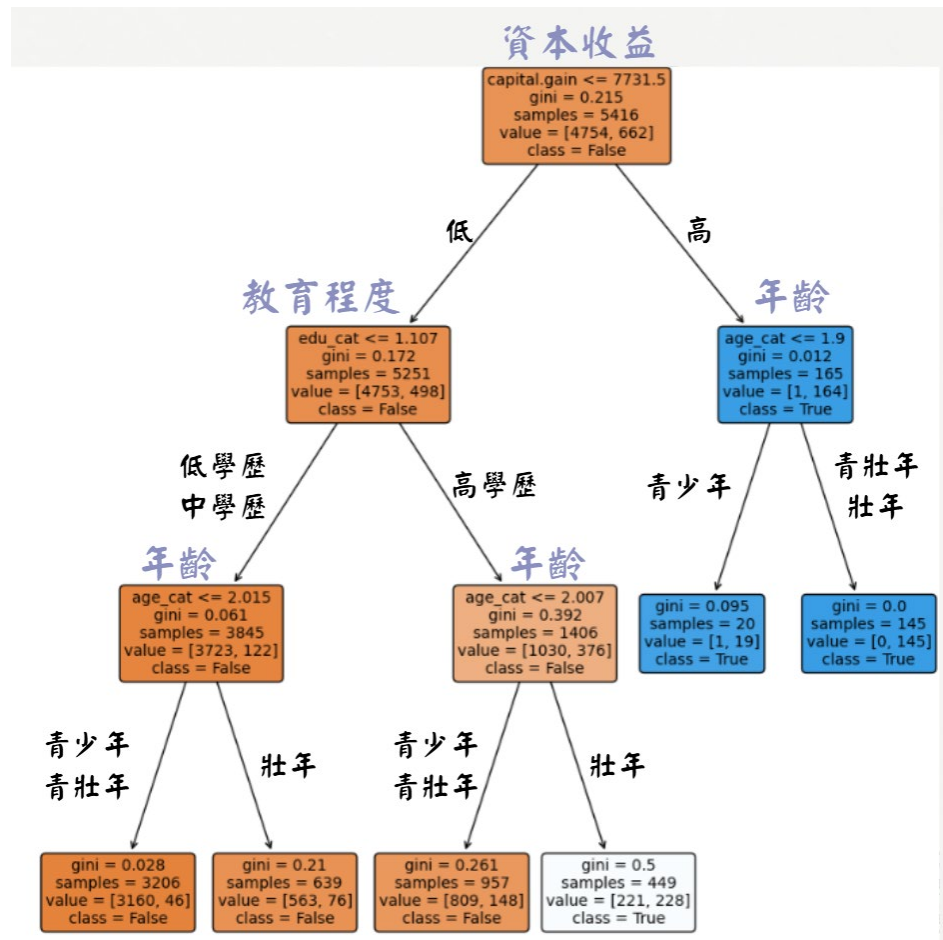


圖 8.1.2 左子樹

8.1.3 右子樹

對於已婚或其他婚姻狀態的族群，模型同樣先以教育程度作為主要分裂依據，顯示學歷在此族群中具有高度判別力。在較高學歷的情況下，模型多預測為高收入，並進一步透過年齡與資本收益細分；反之，低學歷者即使存在資本收益，其預測結果仍較偏向低收入。此結構顯示，已婚族群的收入判斷更強烈依賴教育背景與長期累積的經濟條件。

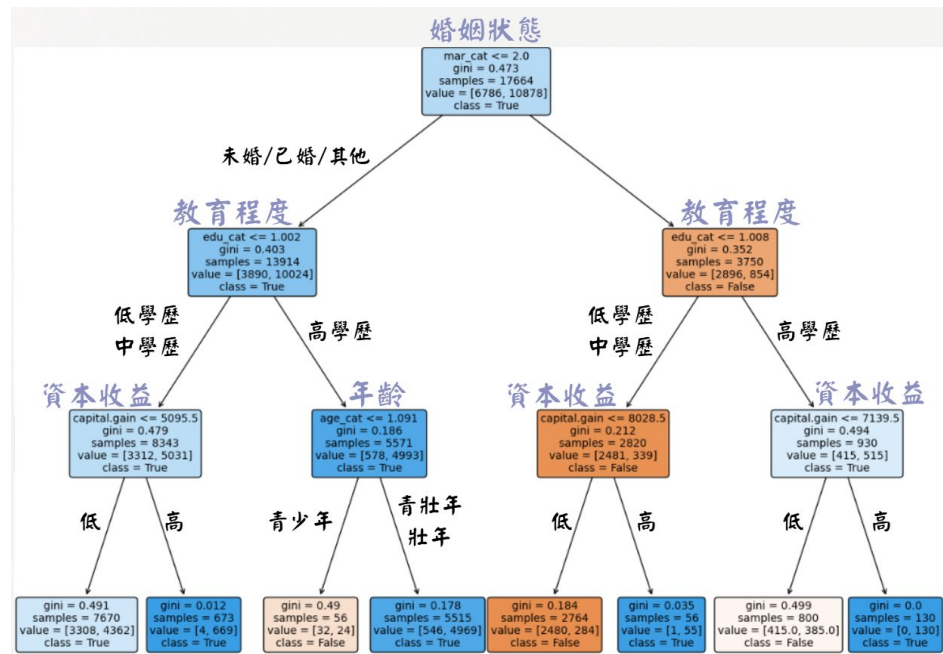


圖 8.1.3 右子樹

8.2 總結

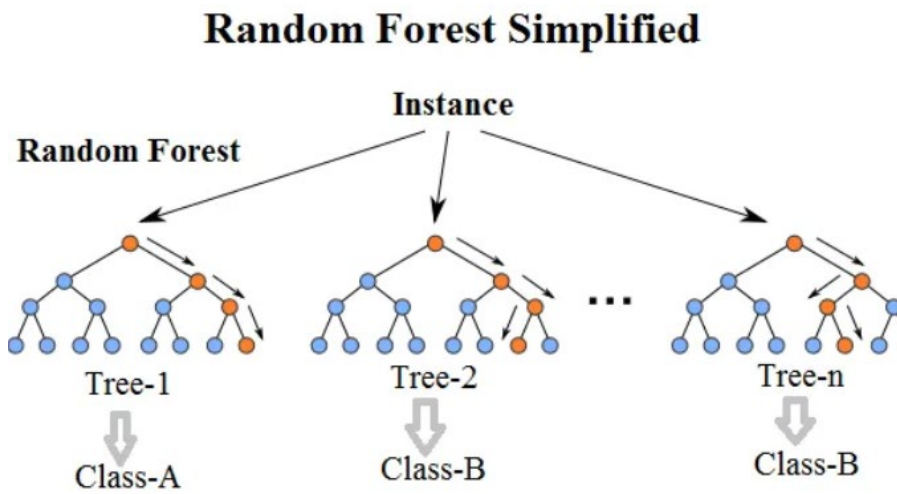
根據決策樹圖而言，婚姻狀態為最關鍵的分裂依據，顯示其對收入分類具有高度判別力。在此基礎上，模型進一步結合教育程度、資本收益與年齡等因素進行細分判斷，使不同族群能被有效區隔。整體而言，未婚族群的收入判斷較依賴個人資本條件與年齡因素，而已婚或其他婚姻狀態者則更明顯受到教育程度與長期經濟條件的影響。

(二)隨機森林

1. 模型介紹: 隨機森林是一種以多顆決策樹為基礎的集成式學習方法，透過 Bootstrap 隨機抽樣的方式自原始資料集中抽取子樣本，以確保模型訓練過程具備隨機性。Bootstrap 的取樣方式具有均勻且可重複抽樣的特性，使每棵決策樹所使用的資料略有差異。

隨機森林模型由多棵決策樹共同組成，最終結果由各決策樹進行整體判斷，有助於降低模型過度擬合的風險。過度擬合是指模型在訓練資料上表現良好，但在測試資料上的誤差明顯增加，常見原因包含資料量不足或過度追求訓練準確

率。與我們所做的另外兩個模型相較之下，隨機森林能透過多模型整合的方式，使預測結果較為穩定。



2. 模型程式碼:

<pre># ===== # 0. 安裝套件 (Colab 才需要) # ===== !pip install pyreadstat seaborn from sklearn.linear_model import LogisticRegression from sklearn.model_selection import StratifiedKFold, cross_val_score from sklearn.metrics import accuracy_score 顯示隱藏的輸出內容 # ===== # 1. 匯入套件 # ===== import pandas as pd import numpy as np import matplotlib.pyplot as plt import seaborn as sns from sklearn.ensemble import RandomForestClassifier from sklearn.preprocessing import LabelEncoder from sklearn.metrics import (confusion_matrix, accuracy_score, classification_report, roc_curve, auc) # ===== # 2. 載入訓練 / 測試資料 # ===== train_path = "/content/train_data.csv" test_path = "/content/test_data.csv" try: train_data = pd.read_csv(train_path) test_data = pd.read_csv(test_path) print("訓練與測試資料載入成功!") except Exception as e: print(f"讀取失敗: {e}") raise e # ===== # 10. 五折交叉驗證 # ===== from sklearn.over_sampling import SMOTE smote = SMOTE(random_state=0) X_train_res, y_train_res = smote.fit_resample(X_train, y_train) from sklearn.model_selection import StratifiedKFold, cross_val_score from sklearn.metrics import accuracy_score, classification_report import numpy as np import pandas as pd cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42) # 五折交叉驗證 accuracy cv_scores = cross_val_score(model, X_train_res, y_train_res, cv=cv, scoring='accuracy') # 只寫標「正確本」報告 report_acc = [] for train_idx, val_idx in cv.split(X_train_res, y_train_res): X_tr = X_train_res.iloc[train_idx] X_val = X_train_res.iloc[val_idx] y_tr = y_train_res[train_idx] y_val = y_train_res[val_idx] model.fit(X_tr, y_tr) y_val_pred = model.predict(X_val) report = classification_report(y_val, y_val_pred, output_dict=True, zero_division=0) # 只存正確率 (label = 1) report_acc.append(report['f1']) # ===== # 3. 指定目標與特徵 # ===== TARGET_COLUMN = "income_bin" FEATURE_COLUMNS = ['mar_cat', 'edu_cat', 'hours_per_week', 'capital_gain', 'age_cat', 'occ_cat']</pre>	<pre># ===== # 3. 指定目標與特徵 # ===== TARGET_COLUMN = "income_bin" FEATURE_COLUMNS = ['mar_cat', 'edu_cat', 'hours_per_week', 'capital_gain', 'age_cat', 'occ_cat']</pre>	<pre># ===== # 7. 缺失值處理 (數值) # ===== for col in X_train.columns: if X_train[col].dtype in [np.float64, np.int64]: median_val = X_train[col].median() X_train[col] = X_train[col].fillna(median_val) X_test[col] = X_test[col].fillna(median_val) # ===== # 8. 儲存類別 One-hot 編碼 # ===== cat_cols = X_train.select_dtypes(include=["object", "category"]).columns cat_cols = pd.get_dummies(X_train, columns=cat_cols, drop_first=True) X_test = pd.get_dummies(X_test, columns=cat_cols, drop_first=True) # 資料欄位 X_train, X_test = X_train.align(X_test, join='left', axis=1, fill_value=0) print("資料欄位處理完成") 資料前處理完成 # ===== # 9. 訓練 Random Forest (保守驗證) # ===== model = RandomForestClassifier(n_estimators=100, max_depth=5, min_samples_leaf=20, random_state=42, n_jobs=-1) print("開始訓練模型...") model.fit(X_train, y_train)</pre>
<pre># ===== # 11. 查看模型性能 - 五折交叉驗證 (含平均) # ===== import matplotlib.pyplot as plt import seaborn as sns import numpy as np from sklearn.model_selection import StratifiedKFold from sklearn.metrics import confusion_matrix, accuracy_score # 合併所有的 train / test 資料 X_combined = np.vstack((X_train, X_test)) y_combined = np.hstack((y_train, y_test)) cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42) # 用來儲存每一折的結果 cm_train_list = [] cm_test_list = [] acc_train_list = [] acc_test_list = [] # 繪出 6 x 2 子圖 (最後一列給平均) fig, axes = plt.subplots(6, 2, figsize=(16, 36)) # 五折交叉驗證 for i, (train_idx, test_idx) in enumerate(cv.split(X_combined, y_combined)): # == 切分資料 == X_train_fold = X_combined[train_idx] X_test_fold = X_combined[test_idx] y_train_fold = y_combined[train_idx] y_test_fold = y_combined[test_idx] # == 訓練模型 == model.fit(X_train_fold, y_train_fold) # == 預測 == y_train_pred = model.predict(X_train_fold) y_test_pred = model.predict(X_test_fold)</pre>	<pre># ===== # 11. 查看模型性能 - 五折交叉驗證 (含平均) # ===== import matplotlib.pyplot as plt import seaborn as sns import numpy as np from sklearn.model_selection import StratifiedKFold from sklearn.metrics import confusion_matrix, accuracy_score # 合併所有的 train / test 資料 X_combined = np.vstack((X_train, X_test)) y_combined = np.hstack((y_train, y_test)) cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42) # 用來儲存每一折的結果 cm_train_list = [] cm_test_list = [] acc_train_list = [] acc_test_list = [] # 繪出 6 x 2 子圖 (最後一列給平均) fig, axes = plt.subplots(6, 2, figsize=(16, 36)) # 五折交叉驗證 for i, (train_idx, test_idx) in enumerate(cv.split(X_combined, y_combined)): # == 切分資料 == X_train_fold = X_combined[train_idx] X_test_fold = X_combined[test_idx] y_train_fold = y_combined[train_idx] y_test_fold = y_combined[test_idx] # == 訓練模型 == model.fit(X_train_fold, y_train_fold) # == 預測 == y_train_pred = model.predict(X_train_fold) y_test_pred = model.predict(X_test_fold)</pre>	
<pre># ===== # 12. ROC Curve (五折交叉驗證) # ===== from sklearn.metrics import roc_curve, auc import numpy as np import matplotlib.pyplot as plt cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42) mean_fpr = np.linspace(0, 1, 100) tprs = [] aucs = [] plt.figure(figsize=(16, 8)) for i, (train_idx, val_idx) in enumerate(cv.split(X_train_res, y_train_res), 1): X_tr = X_train_res.iloc[train_idx] X_val = X_train_res.iloc[val_idx] y_tr = y_train_res[train_idx] y_val = y_train_res[val_idx] model.fit(X_tr, y_tr) y_val_prob = model.predict_proba(X_val)[1, 1] fpr, tpr, _ = roc_curve(y_val, y_val_prob) roc_auc = auc(fpr, tpr) # 畫每一折 ROC plt.plot(fpr, tpr, lw=1, alpha=0.4, label=f'Fold {i} ROC (AUC = {roc_auc:.4f})') # 繪出所有折的平均 ROC interpr_tpr = np.interp(mean_fpr, fpr, tpr) interp_tpr[0] = 0.0 tprs.append(interp_tpr) aucs.append(roc_auc) # ===== # 平均 ROC + 標準差 # ===== mean_tpr = np.mean(tprs, axis=0) mean_tpr[-1] = 1.0 mean_auc = np.mean(aucs) std_auc = np.std(aucs)</pre>	<pre># ===== # 計算平均真值率與平均準確率 # ===== mean_cm_train = np.mean(cm_train_list, axis=0) mean_cm_test = np.mean(cm_test_list, axis=0) mean_acc_train = np.mean(acc_train_list) mean_acc_test = np.mean(acc_test_list) # ===== # 平均 Train Confusion Matrix # ===== sns.heatmap(mean_cm_train, annot=True, fmt='d', cmap='Blues', ax=axess[5, 0]) axess[5, 0].set_title(f'Mean Train (5-Fold CV) Inaccuracy={mean_acc_train:.4f}') axess[5, 0].set_ylabel('Predicted') axess[5, 0].set_xlabel('True') # ===== # 平均 Test Confusion Matrix # ===== sns.heatmap(mean_cm_test, annot=True, fmt='d', cmap='Blues', ax=axess[5, 1]) axess[5, 1].set_title(f'Mean Test (5-Fold CV) Inaccuracy={mean_acc_test:.4f}') axess[5, 1].set_ylabel('Predicted') axess[5, 1].set_xlabel('True') plt.tight_layout() plt.show()</pre>	<pre># ===== # 計算平均真值率與平均準確率 # ===== mean_cm_train = np.mean(cm_train_list, axis=0) mean_cm_test = np.mean(cm_test_list, axis=0) mean_acc_train = np.mean(acc_train_list) mean_acc_test = np.mean(acc_test_list) # ===== # 平均 Train Confusion Matrix # ===== sns.heatmap(mean_cm_train, annot=True, fmt='d', cmap='Blues', ax=axess[5, 0]) axess[5, 0].set_title(f'Mean Train (5-Fold CV) Inaccuracy={mean_acc_train:.4f}') axess[5, 0].set_ylabel('Predicted') axess[5, 0].set_xlabel('True') # ===== # 平均 Test Confusion Matrix # ===== sns.heatmap(mean_cm_test, annot=True, fmt='d', cmap='Blues', ax=axess[5, 1]) axess[5, 1].set_title(f'Mean Test (5-Fold CV) Inaccuracy={mean_acc_test:.4f}') axess[5, 1].set_ylabel('Predicted') axess[5, 1].set_xlabel('True') plt.tight_layout() plt.show()</pre>

```

#
# 平均 ROC - 標準差
#
mean_fpr = np.mean(fpr, axis=0)
mean_tpr[-1] = 1.0
mean_auc = np.mean(auc)
std_auc = np.std(auc)

std_fpr = np.std(fpr, axis=0)

plt.plot(
    mean_fpr,
    mean_tpr,
    color='b',
    label='Mean ROC (AUC = {mean_auc:.4f} ± {std_auc:.4f})',
    lw=2
)

plt.fill_between(
    mean_fpr,
    mean_tpr + std_tpr,
    mean_tpr - std_tpr,
    color='gray',
    alpha=0.2,
    label='±1 std'
)

# Random guess
plt.plot([0, 1], [0, 1], linestyle='-', color='black', label='Random Guess')

plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve (5-fold Cross-Validation)')
plt.legend(loc='lower right')
plt.grid()
plt.show()

# 數據輸出(先的備對標用)
#
print("五折 AUC", auc)
print("五折 AUC", mean_auc)
print("五次標準差", std_auc)
print("平均 ± 標準差: {mean_auc:.4f} ± {std_auc:.4f}")

#
# =====
# 14. 成效報告
# =====

# 重新計算完整的訓練集和測試集預測
# model 是在 step 10 的最後一折交叉驗證中訓練的結果
y_train_pred = model.predict(X_train)
y_test_pred = model.predict(X_test)

# 重新計算完整的訓練集和測試集準確率
acc_train = accuracy_score(y_train, y_train_pred)
acc_test = accuracy_score(y_test, y_test_pred)

print(f"Train Accuracy: {acc_train:.4f}")
print(f"Test Accuracy: {acc_test:.4f}")

print("\n--- Train Classification Report ---")
print(classification_report(y_train, y_train_pred))

print("\n--- Test Classification Report ---")
print(classification_report(y_test, y_test_pred))

```

3. 隨機森林超參數：

樹的數量	100
最大樹深	4
抽資料方式	80/20法則
樹葉節點 (每一個分支的最少樣本數)	20 (資料筆數位於1000~5000之間， 故設置葉節點為20合理)
亂數種子	42

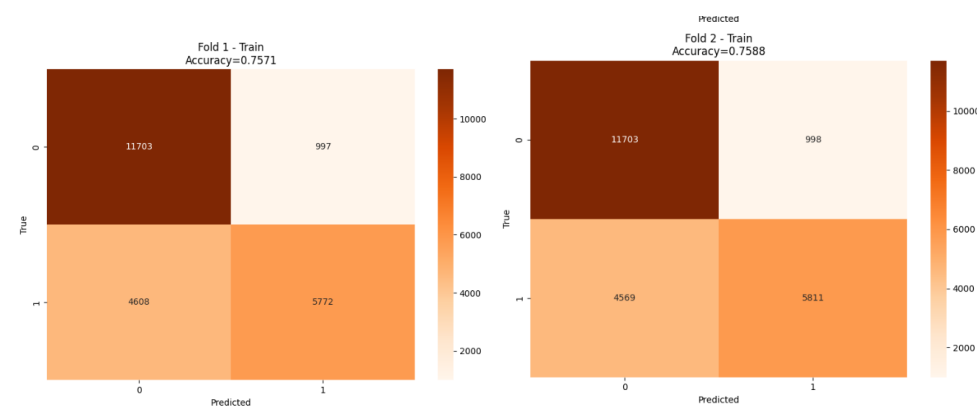
其他參數皆為預設值。

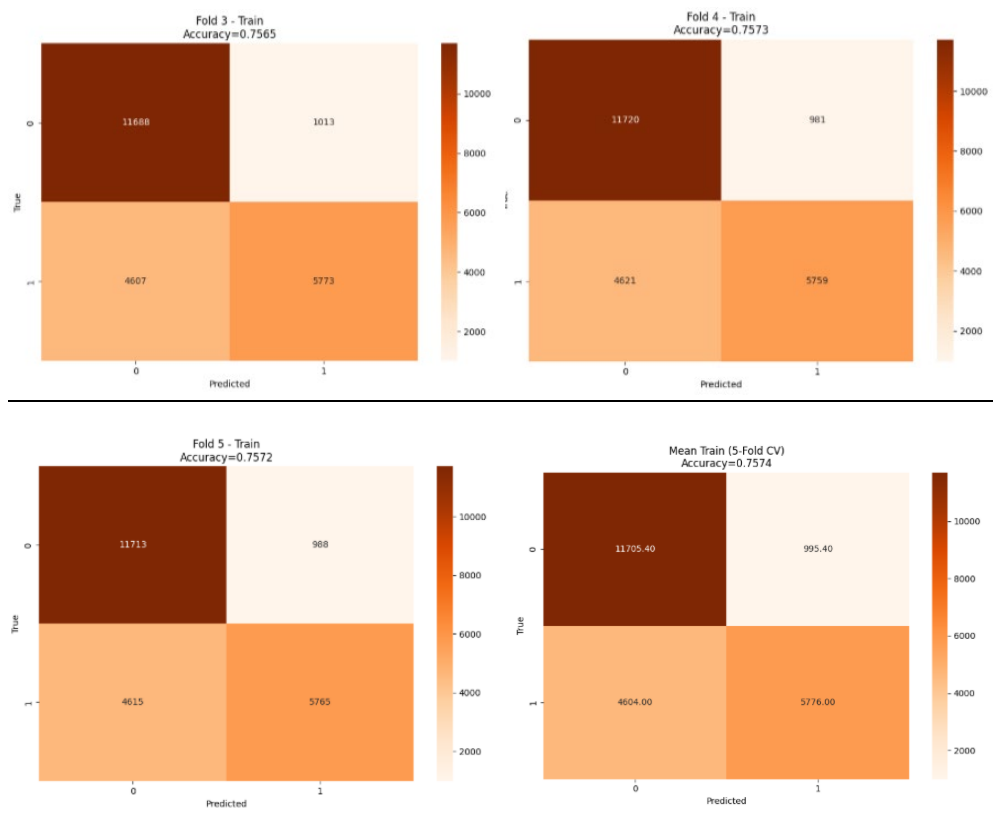
4. 隨機森林其他資料:抽樣時將資料進行下採樣後再來進行隨機抽樣，使訓練集的 income 達成類別平衡， income 有兩種類別，分為 0、1 各抽 11540、11540 筆測試集的 income 中 0、1 個別抽 4336、1435 筆。
5. 利用混淆矩陣檢驗模型:

準確率(Accuracy)		$(TP+TN) / (TP+TN+FP+FN)$
精確率	$Precision = \frac{TP}{TP + FP}$	判斷為真的情況下，有多少真是真的正例
召回率	$Recall = \frac{TP}{TP + FN}$	事實為真的樣本中有幾個是預測正確的
F1分數	$F1-score = 2 \times \frac{Precision \times Recall}{Precision + Recall}$	同時考慮精確率和召回值是兩者間的調和平均

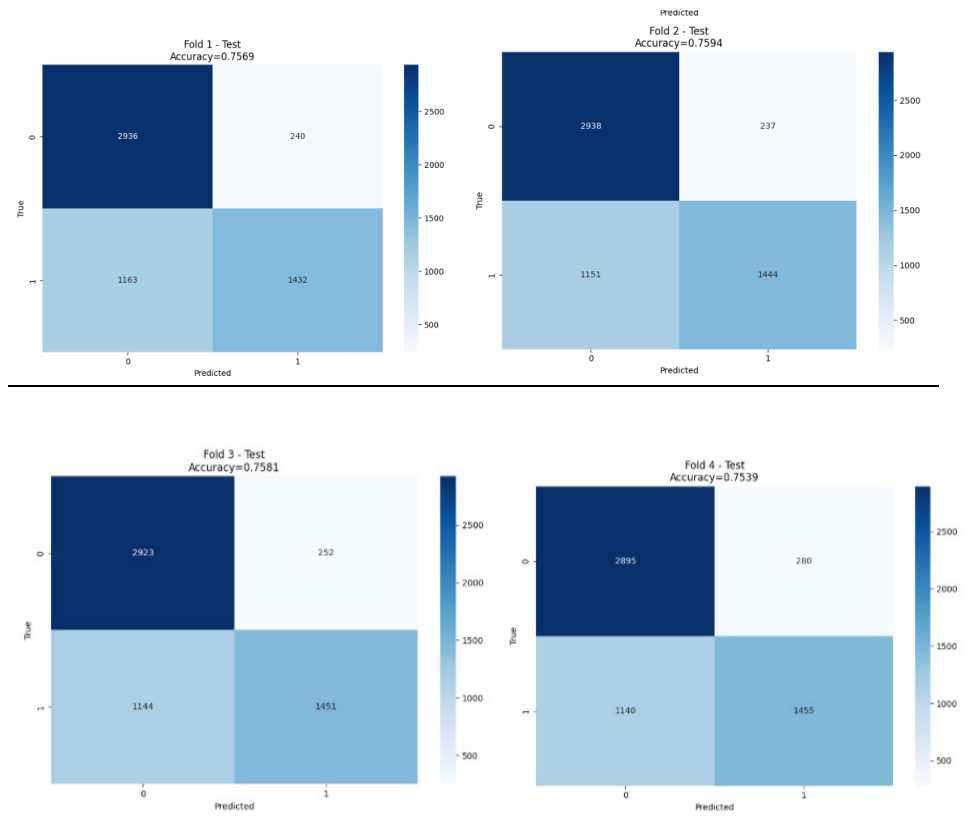
	預測為真	預測為假
實際為真	真陽性 (TP)	偽陰性 (FN)
實際為假	偽陽性 (FP)	真陰性 (TN)

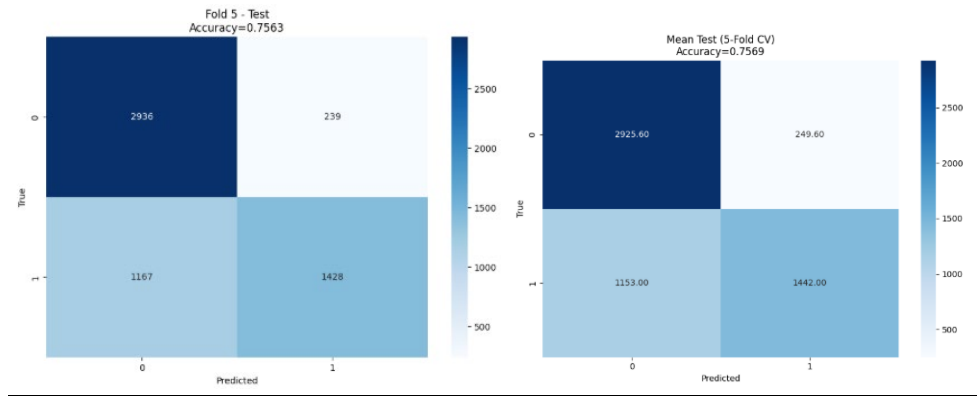
5.1 訓練集的混淆矩陣





5.2 測試集的混淆矩陣

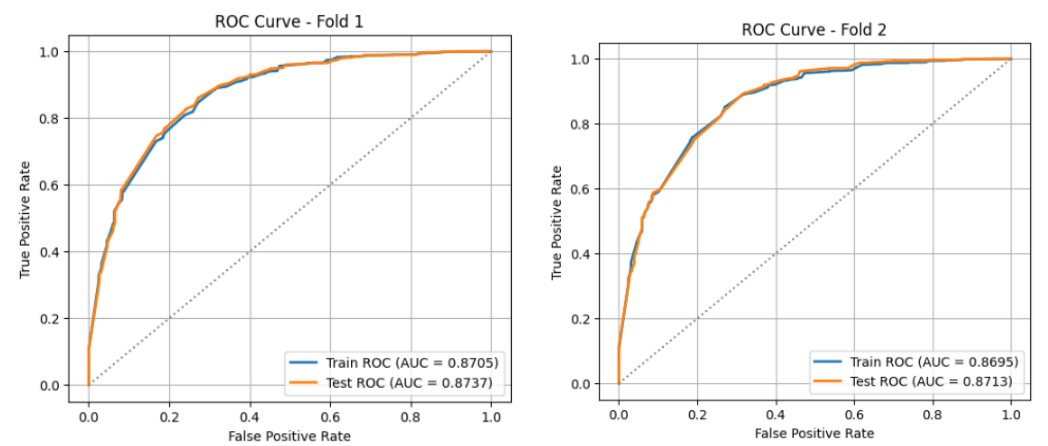


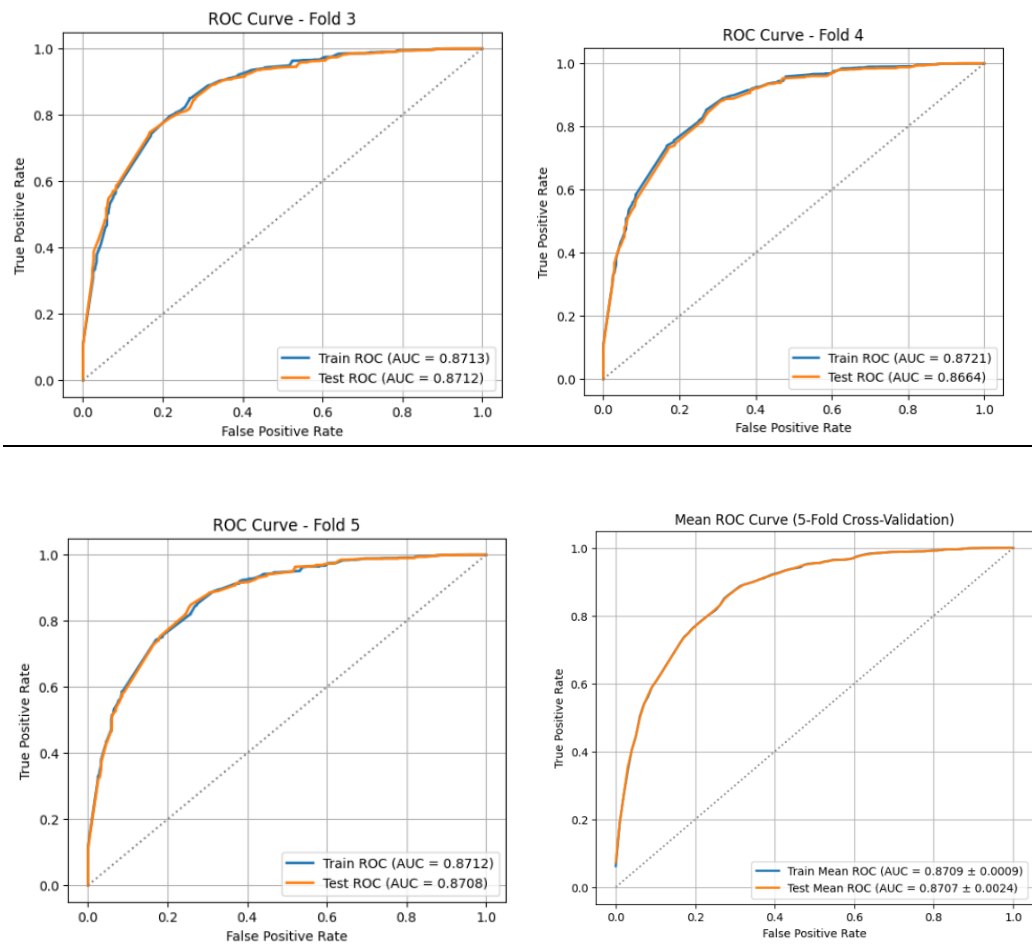


6. ROC 曲線

AUC數值	模型程度
< 0.7	模型弱，需改進
0.7 ~ 0.8	一般
0.8 ~ 0.9	很好
0.9以上	非常優秀

6.1 五折的 ROC 圖





7. 隨機森林結果

7.1. 模型準確度

訓練集約為 **0.7899**，測試集約為 **0.7536**，此結果 100 棵樹集體投票後的準確度每個預測都是由所有 100 棵樹共同決定的

訓練集準確率: 0.7899046793760832
 測試集準確率: 0.7535955640270318

7.2. 五折交叉驗證結果

平均準確率為 **0.7874**

五折交叉驗證準確率: [0.78596187 0.78704506 0.78531196 0.78336222 0.7952773]
平均準確率: 0.7873916811091854

五折交叉驗證平均分類報告:

	positive
precision	0.753206
recall	0.855633
f1-score	0.800928
support	2308.000000

平均標準差為 0.0041

五折偏差 (每折-平均) : [-0.00142981 -0.00034662 -0.00207972 -0.00402946 0.00788562]
五折標準差: 0.004120903903613393

五次交叉驗證標準差分類報告:

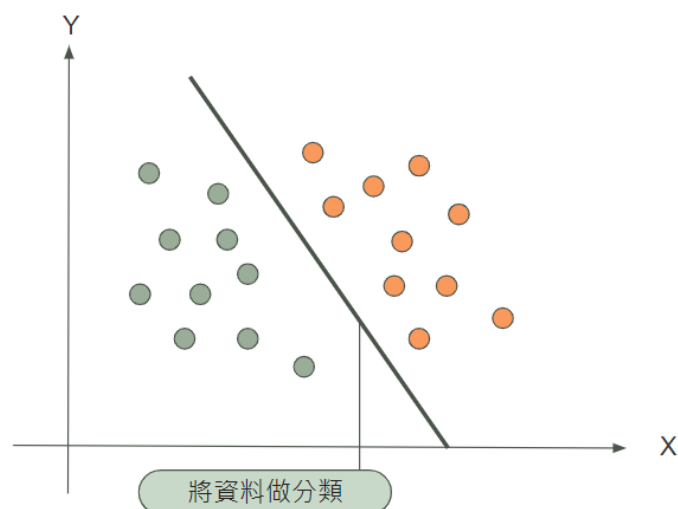
	positive
precision	0.010134
recall	0.018923
f1-score	0.004474
support	0.000000

混淆矩陣約為 0.7874 ± 0.0041

平均 $\pm$ 標準差: $0.7873916811091854 \pm 0.004120903903613393$

(三)羅吉斯回歸

1. 模型介紹:



羅吉斯回歸與線性回歸不同，線性回歸主要用於預測連續型數值，而羅吉斯回歸則常應用於處理分類問題，其目的在於將資料區分為不同類別。本研究，羅吉斯回歸模型的目標是建立一個分類邊界，以

區分高收入與低收入兩類族群。 在建構模型之前，需先定義特徵 (X) 與標籤 (Y)。我們以「收入」作為標籤，其中收入是否大於或等於 50K 作為分類依據；特徵則包含年齡、教育程度等相關變數。羅吉斯回歸模型以勝算比 (Odds Ratio) 作為基礎進行計算，勝算比為收入大於或等於 50K 的發生機率與收入小於 50K 的發生機率之比。透過對勝算比取對數後，再經由 Sigmoid 函數轉換，將預測結果限制在 0 至 1 之間，作為發生機率的表示。最終，當預測機率大於或等於 0.5 時，判定為高收入 (1)，反之則判定為低收入 (0)。

2. 模型建立:

建構羅吉斯回歸的模型是使用 Scikit-learn 的 LogisticRegression，訓練集和測試集分為 80% 和 20%，迭代次數設為 2000 代，優化器選擇 saga，再使用 fit 去擬合訓練集資料。下圖為程式碼。

```
# =====
# 1 套件
# =====
'pip install pyreadstat
import pandas as pd
import numpy as np
import pyreadstat
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix, roc_curve, auc

# =====
# 2 讀取資料
# =====
train_df, _ = pyreadstat.read_sav("train_balanced.sav")
test_df, _ = pyreadstat.read_sav("test.sav")

feature_cols = [
    "work_cat", "edu_cat", "mar_cat", "occ_cat", "rel_cat",
    "race_cat", "sex_cat", "nat_cat", "educ_cat", "age_cat",
    "capital_gain", "capital_loss", "hours_per_week"
]

X_train = train_df[feature_cols]
y_train = train_df["income_bin"]

X_test = test_df[feature_cols]
y_test = test_df["income_bin"]

# =====
# 3 模型 Pipeline
# =====
pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("logistic", LogisticRegression(
        solver="saga", max_iter=2000, tol=1e-3,
        class_weight="balanced", n_jobs=-1
    ))
])
```

```
# =====
# 五折交叉验证
# =====
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

train_acc_list = []
test_acc_list = []
cv_acc_list = []

train_reports = []
test_reports = []

roc_train_list = []
roc_val_list = []
roc_test_list = []

fold = 1
for train_idx, val_idx in cv.split(X_train, y_train):
    X_tr, X_val = X_train.iloc[train_idx], X_train.iloc[val_idx]
    y_tr, y_val = y_train.iloc[train_idx], y_train.iloc[val_idx]

    pipeline.fit(X_tr, y_tr)

    # 预测
    y_tr_pred = pipeline.predict(X_tr)
    y_val_pred = pipeline.predict(X_val)
    y_test_pred = pipeline.predict(X_test)

    y_tr_prob = pipeline.predict_proba(X_tr)[:,1]
    y_val_prob = pipeline.predict_proba(X_val)[:,1]
    y_test_prob = pipeline.predict_proba(X_test)[:,1]

    # Accuracy
    acc_train = accuracy_score(y_tr, y_tr_pred)
    acc_val = accuracy_score(y_val, y_val_pred)
    acc_test = accuracy_score(y_test, y_test_pred)

    train_acc_list.append(acc_train)
    test_acc_list.append(acc_test)
    cv_acc_list.append(acc_val)

    # 分类报告
    train_reports.append(classification_report(y_tr, y_tr_pred, output_dict=True))
    test_reports.append(classification_report(y_test, y_test_pred, output_dict=True))

    # ROC
    fpr_tr, tpr_tr, _ = roc_curve(y_tr, y_tr_prob)
    fpr_val, tpr_val, _ = roc_curve(y_val, y_val_prob)
    fpr_te, tpr_te, _ = roc_curve(y_test, y_test_prob)

    roc_train_list.append((fpr_tr, tpr_tr))
    roc_val_list.append((fpr_val, tpr_val))
    roc_test_list.append((fpr_te, tpr_te))

    roc_auc_train = auc(fpr_tr, tpr_tr)
    roc_auc_val = auc(fpr_val, tpr_val)
    roc_auc_test = auc(fpr_te, tpr_te)
```

```
fig, axes = plt.subplots(2,3, figsize=(18,10))
```

```
# 混淆矩阵
```

```
sns.heatmap(confusion_matrix(y_tr, y_tr_pred), annot=True, fmt='d', ax=axes[0,0], cmap='Blues')
axes[0,0].set_title(f'Fold {fold} Train Confusion Matrix')
axes[0,0].set_xlabel('Predicted')
axes[0,0].set_ylabel('Actual')
```

```
sns.heatmap(confusion_matrix(y_val, y_val_pred), annot=True, fmt='d', ax=axes[0,1], cmap='Greens')
axes[0,1].set_title(f'Fold {fold} Validation Confusion Matrix')
axes[0,1].set_xlabel('Predicted')
axes[0,1].set_ylabel('Actual')
```

```
sns.heatmap(confusion_matrix(y_test, y_test_pred), annot=True, fmt='d', ax=axes[0,2], cmap='Oranges')
axes[0,2].set_title(f'Fold {fold} Test Confusion Matrix')
axes[0,2].set_xlabel('Predicted')
axes[0,2].set_ylabel('Actual')
```

```
train_acc_mean = np.mean(train_acc_list)
train_acc_std = np.std(train_acc_list)
test_acc_mean = np.mean(test_acc_list)
test_acc_std = np.std(test_acc_list)
cv_acc_mean = np.mean(cv_acc_list)
cv_acc_std = np.std(cv_acc_list)
```

```
print("\n==== Average Accuracy =====")
print(f"Train: {train_acc_mean:.4f} ± {train_acc_std:.4f}")
print(f"Test : {test_acc_mean:.4f} ± {test_acc_std:.4f}")
print(f"CV : {cv_acc_mean:.4f} ± {cv_acc_std:.4f}")
```

```
# 偏差 = 每折 - 平均
```

```
train_acc_dev = np.array(train_acc_list) - train_acc_mean
test_acc_dev = np.array(test_acc_list) - test_acc_mean
cv_acc_dev = np.array(cv_acc_list) - cv_acc_mean
```

```
print("\n==== Accuracy Deviation per Fold =====")
```

```
for i in range(5):
    print(f'Fold {i+1}: Train={train_acc_dev[i]:+.4f}, Test={test_acc_dev[i]:+.4f}, CV={cv_acc_dev[i]:+.4f}')
```

```
def avg_classification_report(reports):
    keys = [k for k in reports[0].keys() if k not in ['accuracy', 'macro avg', 'weighted avg']]
    avg_report = {}
    std_report = {}
    for k in keys:
        avg_report[k] = 0
        std_report[k] = 0
        for metric in ['precision', 'recall', 'f1-score', 'support']:
            values = [r[k][metric] for r in reports]
            avg_report[k][metric] = np.mean(values)
            std_report[k][metric] = np.std(values)
    return avg_report, std_report

avg_train_report, std_train_report = avg_classification_report(train_reports)
avg_test_report, std_test_report = avg_classification_report(test_reports)

def print_avg_report(avg_report, std_report):
    for k in avg_report.keys():
        print(f'\nClass {k}:')
        for metric in ['precision', 'recall', 'f1-score', 'support']:
            mean = avg_report[k][metric]
            std = std_report[k][metric]
            print(f'{metric}: {mean:.4f} ± {std:.4f}')

print("\n==== Average Classification Report - Train (Mean ± Std) =====")
print_avg_report(avg_train_report, std_train_report)

print("\n==== Average Classification Report - Test (Mean ± Std) =====")
print_avg_report(avg_test_report, std_test_report)
```

```

===== Average Accuracy =====
Train: 0.7921 ± 0.0015
Test : 0.7929 ± 0.0009
CV : 0.7918 ± 0.0050

===== Accuracy Deviation per Fold =====
Fold 1: Train=+0.0015, Test=+0.0009, CV=+0.0052
Fold 2: Train=+0.0020, Test=-0.0003, CV=-0.0041
Fold 3: Train=-0.0010, Test=-0.0013, CV=+0.0046
Fold 4: Train=-0.0011, Test=-0.0005, CV=+0.0018
Fold 5: Train=+0.0016, Test=+0.0011, CV=-0.0075

===== Average Classification Report - Train (Mean ± Std) =====

Class 0.0:
precision: 0.7961 ± 0.0017
recall: 0.7854 ± 0.0015
f1-score: 0.7907 ± 0.0015
support: 9232.0000 ± 0.0000

Class 1.0:
precision: 0.7883 ± 0.0014
recall: 0.7989 ± 0.0019
f1-score: 0.7935 ± 0.0015
support: 9232.0000 ± 0.0000

===== Average Classification Report - Test (Mean ± Std) =====

Class 0.0:
precision: 0.9232 ± 0.0007
recall: 0.7900 ± 0.0016
f1-score: 0.8514 ± 0.0008
support: 4336.0000 ± 0.0000

Class 1.0:
precision: 0.5582 ± 0.0015
recall: 0.8014 ± 0.0023
f1-score: 0.6580 ± 0.0010
support: 1435.0000 ± 0.0000

# ROC 曲線
axes[1,0].plot(fpr_tr, tpr_tr, color='blue', lw=2, label=f'AUC = {roc_auc_train:.3f}')
axes[1,0].plot([0,1],[0,1], linestyle='--', color='gray')
axes[1,0].set_title(f'Fold {fold} Train ROC')
axes[1,0].set_xlabel('FPR')
axes[1,0].set_ylabel('TPR')
axes[1,0].legend(loc='lower right')

axes[1,1].plot(fpr_val, tpr_val, color='green', lw=2, label=f'AUC = {roc_auc_val:.3f}')
axes[1,1].plot([0,1],[0,1], linestyle='--', color='gray')
axes[1,1].set_title(f'Fold {fold} Validation ROC')
axes[1,1].set_xlabel('FPR')
axes[1,1].set_ylabel('TPR')
axes[1,1].legend(loc='lower right')

axes[1,2].plot(fpr_te, tpr_te, color='orange', lw=2, label=f'AUC = {roc_auc_test:.3f}')
axes[1,2].plot([0,1],[0,1], linestyle='--', color='gray')
axes[1,2].set_title(f'Fold {fold} Test ROC')
axes[1,2].set_xlabel('FPR')
axes[1,2].set_ylabel('TPR')
axes[1,2].legend(loc='lower right')

plt.tight_layout()
plt.show()

print(f"\n===== Fold {fold} =====")
print(f"Train Accuracy: {acc_train:.4f}, Validation Accuracy: {acc_val:.4f}, Test Accuracy: {acc_test:.4f}")
fold += 1

```

```

# =====
# 7 平均 ROC 曲線 ± 標準差
# =====
mean_fpr = np.linspace(0, 1, 100)
tpr_train_interp = np.array([np.interp(mean_fpr, fpr, tpr) for fpr, tpr in roc_train_list])
tpr_val_interp = np.array([np.interp(mean_fpr, fpr, tpr) for fpr, tpr in roc_val_list])
tpr_test_interp = np.array([np.interp(mean_fpr, fpr, tpr) for fpr, tpr in roc_test_list])

mean_tpr_train = tpr_train_interp.mean(axis=0)
std_tpr_train = tpr_train_interp.std(axis=0)
mean_tpr_val = tpr_val_interp.mean(axis=0)
std_tpr_val = tpr_val_interp.std(axis=0)
mean_tpr_test = tpr_test_interp.mean(axis=0)
std_tpr_test = tpr_test_interp.std(axis=0)

plt.figure(figsize=(18, 5))

plt.subplot(1, 3, 1)
plt.plot(mean_fpr, mean_tpr_train, color="orange", label="Mean ROC Train")
plt.fill_between(mean_fpr, mean_tpr_train-std_tpr_train, mean_tpr_train+std_tpr_train, color="orange", alpha=0.2)
plt.plot([0, 1], [0, 1], linestyle="--", color="gray")
plt.title("Mean ROC Curve - Train")
plt.xlabel("FPR")
plt.ylabel("TPR")
plt.legend()

plt.subplot(1, 3, 2)
plt.plot(mean_fpr, mean_tpr_val, color="green", label="Mean ROC Validation")
plt.fill_between(mean_fpr, mean_tpr_val-std_tpr_val, mean_tpr_val+std_tpr_val, color="green", alpha=0.2)
plt.plot([0, 1], [0, 1], linestyle="--", color="gray")
plt.title("Mean ROC Curve - Validation")
plt.xlabel("FPR")
plt.ylabel("TPR")
plt.legend()

plt.subplot(1, 3, 3)
plt.plot(mean_fpr, mean_tpr_test, color="blue", label="Mean ROC Test")
plt.fill_between(mean_fpr, mean_tpr_test-std_tpr_test, mean_tpr_test+std_tpr_test, color="blue", alpha=0.2)
plt.plot([0, 1], [0, 1], linestyle="--", color="gray")
plt.title("Mean ROC Curve - Test")
plt.xlabel("FPR")
plt.ylabel("TPR")
plt.legend()

plt.tight_layout()
plt.show()

```

```

# =====
# 8 特徵重要性 & Odds Ratio
# =====
coef_df = pd.DataFrame({
    'feature': feature_cols,
    'coefficient': pipeline.named_steps['logistic'].coef_[0]
})
coef_df['odds_ratio'] = np.exp(coef_df['coefficient'])
coef_df = coef_df.sort_values(by='coefficient', key=abs, ascending=False)
print("\n特徵重要性與勝算比 (Odds Ratio):\n", coef_df)

```

由於發現測試集準確率高於訓練集、不符合常理，因此推斷可能有

以下原因：

- 模型的正則化效果/ 由於 Scikit-learn 預設應用 L2 正則化，防止過度擬合、提高泛化能力，使得模型在訓練集上的表現略差、測試集上表現更好。因此在模型建立時選擇將 `penalty='none'` 不使用正則化技術。
- 資料集分佈的差異/ 測試集和訓練集的數據分佈有所不同，也就是測試集中的數據特徵與訓練集相比可能更簡單或者噪音較少，表示特徵數據中可能包含過多離群值，導致測試集由於選取更少的資料因而較低的可能含有影響模型準確率的數值。

3. 混淆矩陣：

3.1 名詞解釋

混淆矩陣 (Confusion Matrix) 是評估分類模型性能的常用工具。圖片展示了在 5-fold 交叉驗證 (Cross-Validation) 下，訓練集 (Train)、驗證集 (Validation)、以及測試集 (Test) 的混淆矩陣結果。

3.2 假設說明

實際值 (Actual)：真實的收入類別

預測值 (Predicted)：模型預測的收入類別

類別定義：

0：低收入

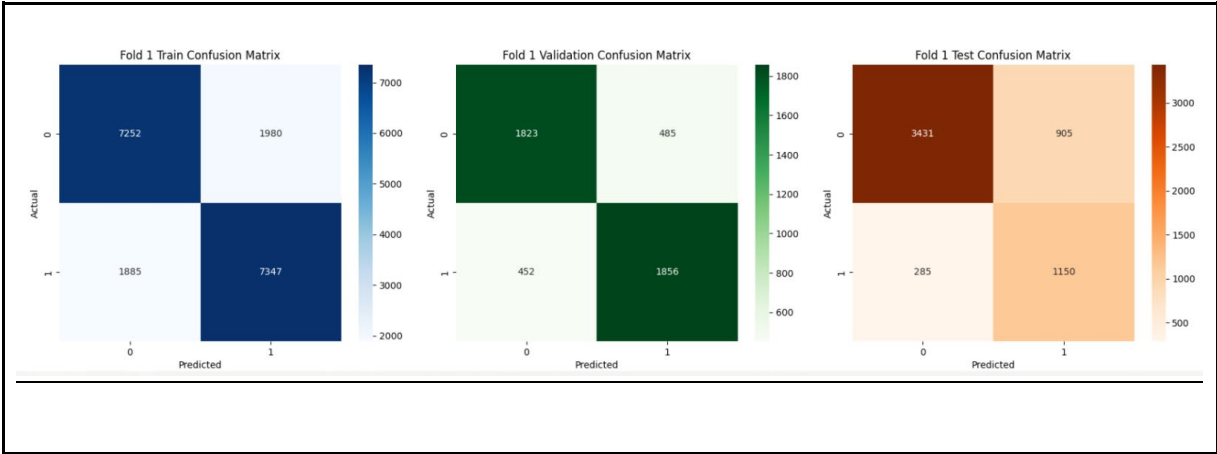
1：高收入

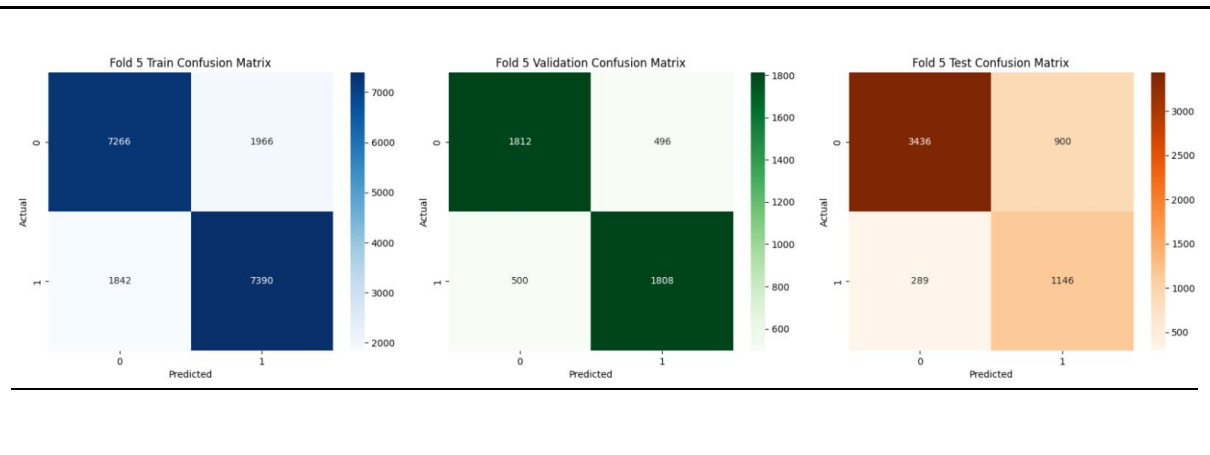
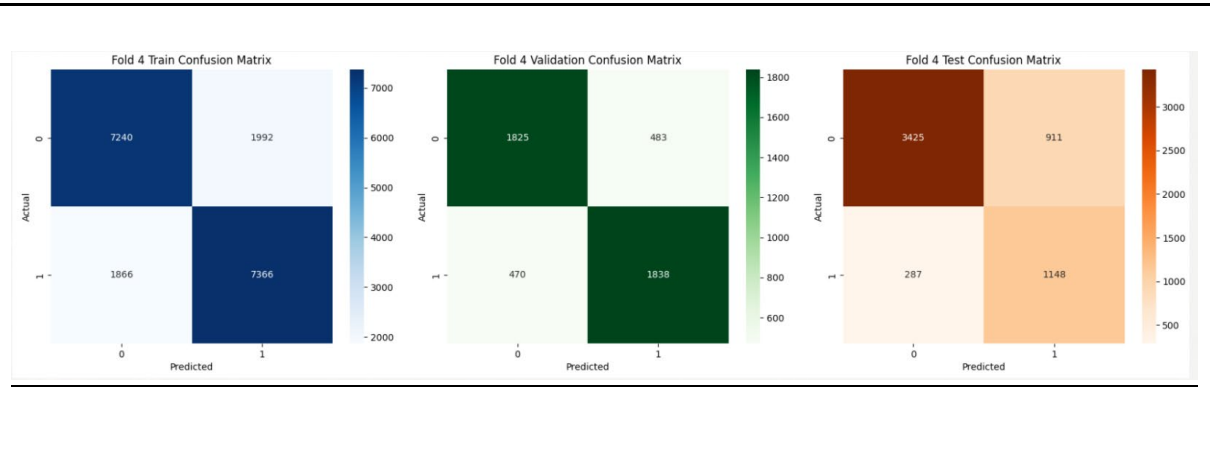
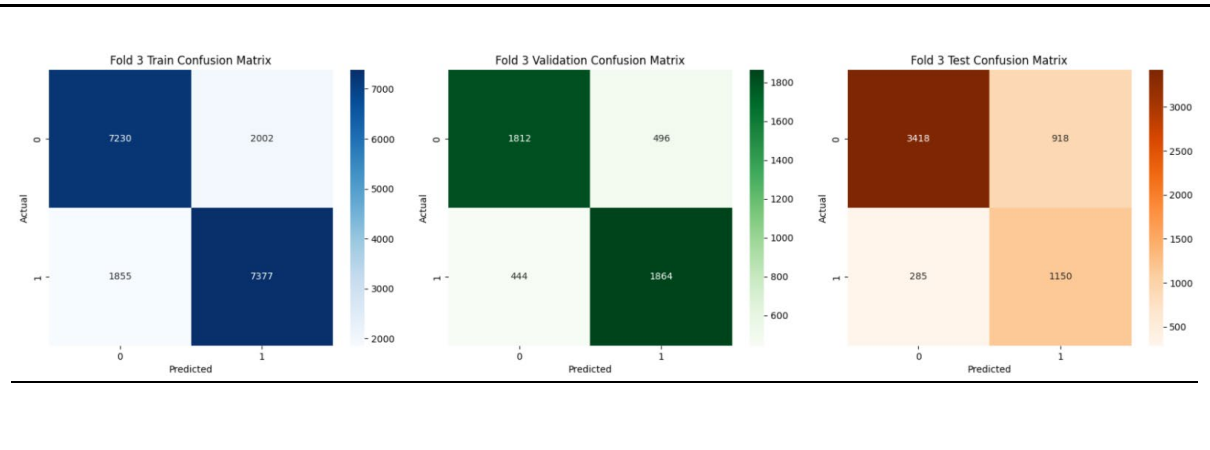
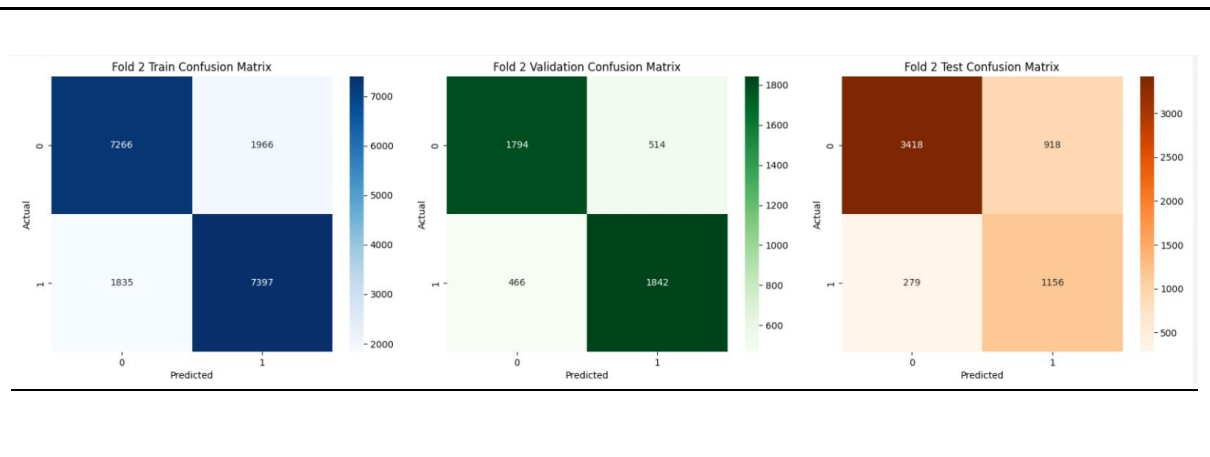
實際值 \ 預測值	預測為 0 (低收入)	預測為 1 (高收入)
實際為 0 (低收入)	TN (True Negative)	FP (False Positive)
實際為 1 (高收入)	FN (False Negative)	TP (True Positive)

3.3 模型效能指標對應表

指標名稱	計算方式	代表意義	與本研究關聯
Accuracy（準確率）	$(TP + TN) / \text{全部樣本}$	整體預測正確比例	受樣本不平衡影響，需搭配其他指標解讀
Precision（精確率）	$TP / (TP + FP)$	預測為高收入者中，真正為高收入的比例	高 precision 代表誤判高收入者較少
Recall（召回率）	$TP / (TP + FN)$	實際高收入者中，被正確抓出的比例	低 recall 代表模型漏判高收入族群
F1-score	$2 \times (Precision \times Recall) / (Precision + Recall)$	精確率與召回率的平衡指標	適合評估類別不平衡問題

3.4 模型訓練結果





3.4 訓練集分類報告分析重點：

<u>類別</u>	<u>Precision</u> <u>(精確率)</u>	<u>Recall</u> <u>(召回率)</u>	<u>F1-Score</u> <u>(F1 分數)</u>	<u>Support</u> <u>(樣本數)</u>
<u>低收入</u>	<u>0.7978</u>	<u>0.7870</u>	<u>0.7924</u>	<u>9232</u>
<u>高收入</u>	<u>0.7899</u>	<u>0.8005</u>	<u>0.7951</u>	<u>9232</u>
總體 Accuracy	0.7922			

1. 性能平衡：由於數據集是平衡的，模型對兩個類別（0.0 和 1.0）的 Precision、Recall 和 F1-Score 都非常接近，均維持在 0.79 左右。
2. 總體精度：總體準確率（Accuracy）為 0.7938。
3. 結論：這是一個相對均衡的性能，沒有明顯偏向任一類別進行預測。這也表明模型在訓練集上的表現中規中矩。

3.5 測試集分類報告分析重點：

<u>類別</u>	<u>Precision (精確率)</u>	<u>Recall (召回率)</u>	<u>F1-Score</u> <u>(F1 分數)</u>	<u>Support</u> <u>(樣本數)</u>
0.0 (多數類)	0.9224	0.7924	0.8525	4336
1.0 (少數類)	0.5601	0.7986	0.6584	1435

總體 Accuracy	0.7929			
----------------	--------	--	--	--

1. 總體精度與訓練集相似：總體準確率為 0.7940，與訓練集的 0.7938 非常接近。這表示模型具有良好的泛化能力，在未見過的測試數據上維持了原有的總體精度。
2. 類別性能差異巨大（不平衡影響）：
 - 低收入=0（多數類）：具有極高的 Precision (0.9224)，這意味著模型預測為 0.0 的樣本中，有超過 92% 是正確的。但 Recall (0.7924) 稍低。
 - 高收入=1（少數類）：Recall (0.7986) 很高，表示模型成功識別了近 80% 的實際類別 1 樣本（涵蓋性高）。Precision 卻很低 (0.5601)，這是主要問題。這意味著模型預測為 1.0 的樣本中，只有約 56% 是正確的，有大量的類別 0 樣本被錯誤地識別為類別 1（偽陽性 FP 偏高）。
3. F1-Score 差異：類別 0 的 F1-Score (0.8525) 遠高於類別 1 的 F1-Score (0.6584)，清晰地顯示了模型在少數類上的表現較差。

3.6 總體比較與結論：

<u>評估項目</u>	<u>訓練集 (平衡數據)</u>	<u>測試集 (不平衡數據)</u>	<u>差異與判讀</u>
總體精度 (Accuracy)	0.7938	0.7940	精度穩定，泛化能力良好。
低收入 Precision	0.7978	0.9224	在不平衡數據上，模型預測多數類 (0.0) 的準確性大幅提升。

高收入 Precision	0.7899	0.5601	在不平衡數據上，模型預測少數類（1.0）的準確性大幅下降。
高收入 Recall	0.8005	0.7986	Recall 保持穩定，表示模型識別少數類的能力未受不平衡數據影響。

1. 模型是穩定的：模型的總體準確率在訓練集和測試集上幾乎沒有變化，表明模型結構穩定。
2. 不平衡的影響：當模型面對不平衡的測試集時，它的預測傾向發生了變化：
 - 為了維持對少數類（1.0）的高 Recall，模型必須更積極地預測 1.0。
 - 這種積極的預測導致了大量的誤報，即將多數類（0.0）的樣本錯誤分類為 1.0，從而使得類別 1 的 Precision 嚴重下降。
 - 結論：必須根據具體的業務需求來判斷這種權衡是否可接受。如果目標是盡可能找出所有類別 1 的樣本（高 Recall），那麼當前的模型是有效的；但如果要求預測結果是高可信度的（高 Precision），則模型性能較差。
3. 穩定性與泛化能力
 - 極低的標準差：測試集準確率的標準差（ ≈ 0.0009 ）非常低，這表明模型在不同的測試子集上表現極為穩定。訓練集和驗證集的標準差也相對較低。
 - 精度一致性：三個數據集（Train: 0.7922, Validation: 0.7918, Test: 0.7929）的平均準確率高度一致，均約為 79.2%。結論：模型沒有出現過度擬合（Overfitting）或擬合不足（Underfitting）的現象，具有優秀的穩定性和良好的泛化能力。
4. 測試集優於訓練集（Validation）
 - Validation 波動較大：驗證集準確率的標準差（0.0051）略高於訓練集（0.0016）和測試集（0.0009），顯示模型在處理不同驗證折時的表現波動稍大。
 - Test Accuracy 略高：測試集的平均準確率（0.7929）略高於訓練集（0.7922）和驗證集（0.7918）。

- 解釋： 這與模型建立部分所述的正則化調整（即使設定 `penalty='none'`，內建的優化器或資料分佈仍可能產生微小影響）或資料集分佈差異的推論相符，但由於差異極小，整體上仍被視為一個性能均衡的模型。

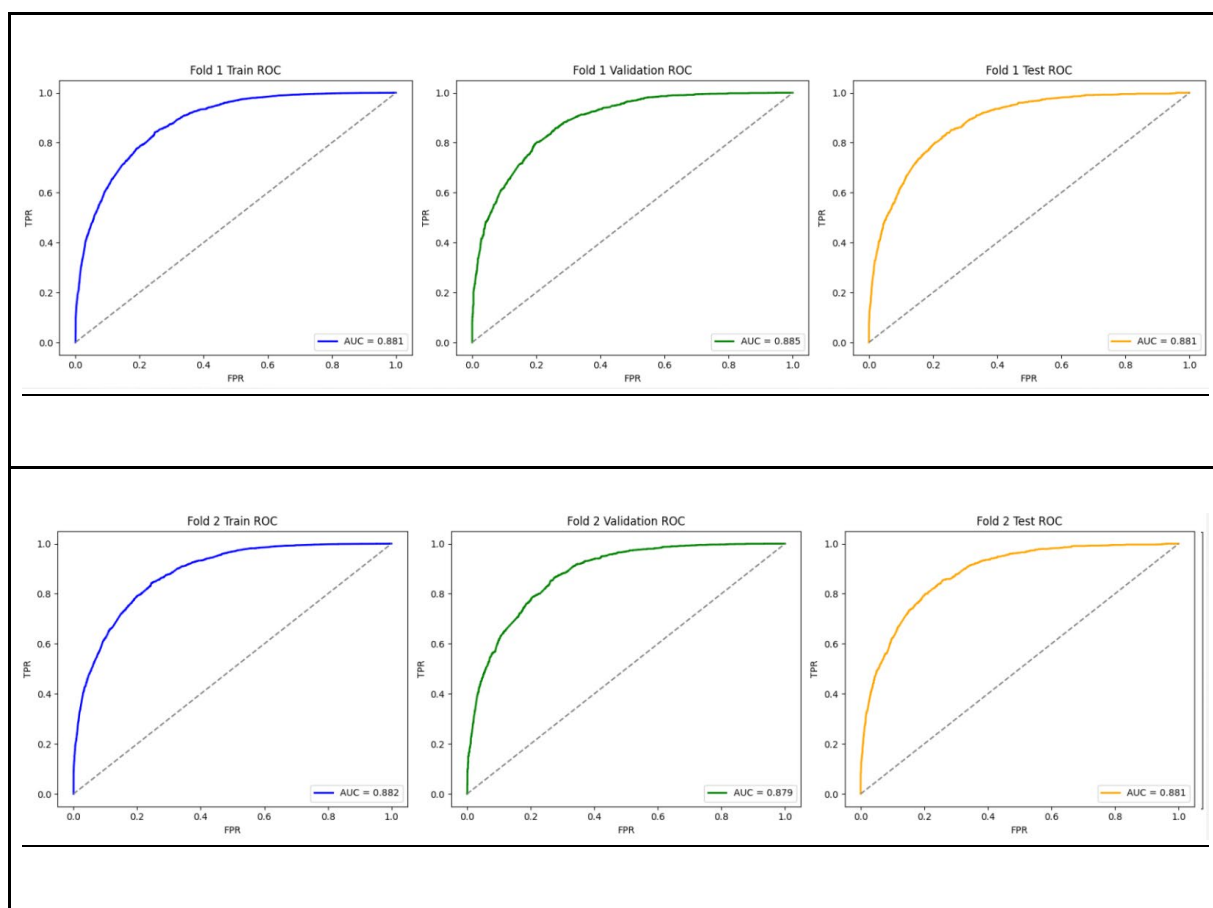
4. ROC 曲線：

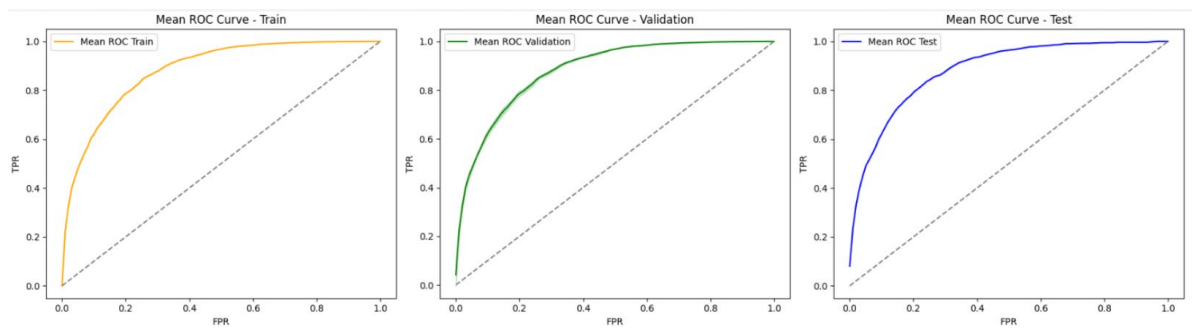
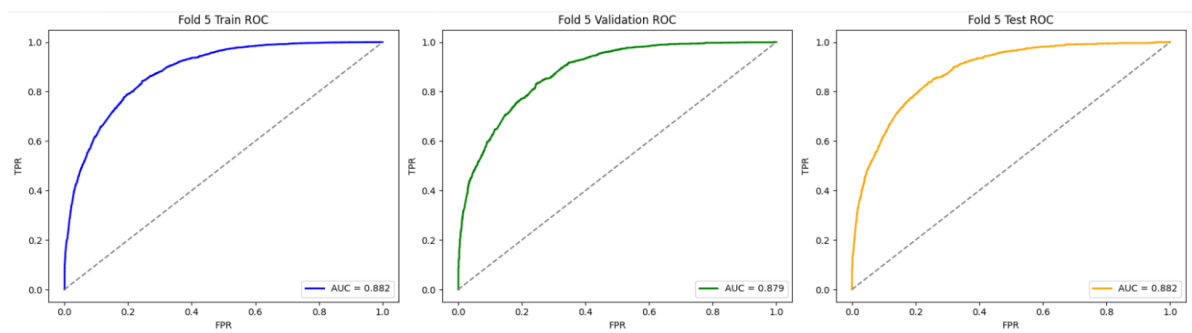
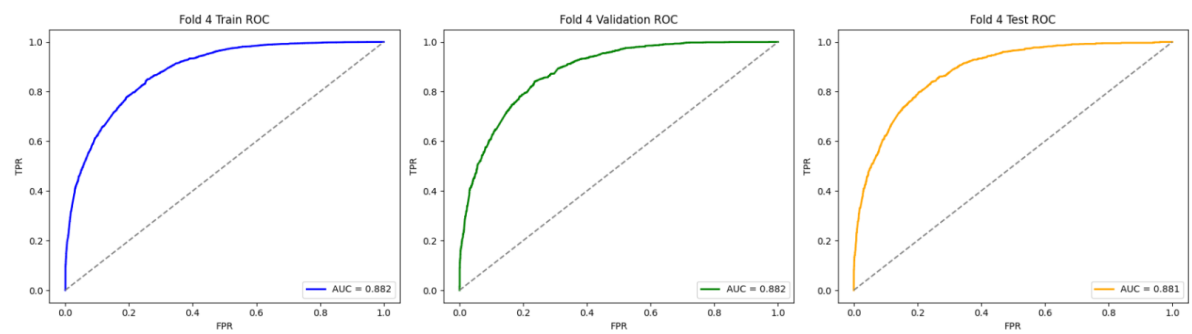
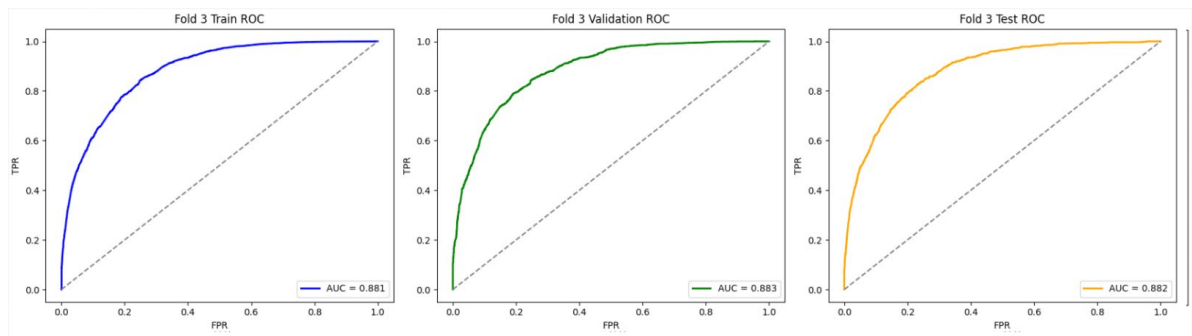
4.1 名詞解釋

ROC 曲線 (Receiver Operating Characteristic Curve) 及其曲線下面積 AUC (Area Under the Curve) 用於評估分類模型在不同分類閾值下的性能。

- Y 軸 (TPR): 真陽性率 (True Positive Rate) 或召回率 (Recall)
- X 軸 (FPR): 偽陽性率 (False Positive Rate)
- 曲線越靠近左上角，AUC 值越高，表示模型性能越好。

4.2 模型結果





4.3 總體比較與結論：

1. 卓越的區分能力 (Mean AUC)

- 平均表現：模型的平均 AUC 值在訓練集、驗證集和測試集上均穩定維持在 0.881 左右。
- 判讀：AUC 數值高達 0.88，表示該羅吉斯回歸模型在隨機選擇一對高收入與低收入樣本時，能正確判斷出高收入者的機率超過 88%。這說明模型對於兩個類別具有高度且有效的區分能力。

2. 極高的穩定性 (Standard Deviation, Std)

- 測試集穩定性：測試集 AUC 的標準差僅有 0.000071。
- 判讀：這個極低的標準差值是模型品質的重要指標，證明模型在五次不同的資料切割和測試中，其性能表現幾乎沒有波動。
- 結論：羅吉斯回歸模型不僅性能優異，更具有極高的穩定性和可靠的泛化能力。

3. 泛化能力評估

- 數據一致性：Train AUC (0.8819)、Validation AUC (0.8812) 和 Test AUC (0.8815) 的平均數值彼此之間非常接近。
- 結論：模型在訓練數據上達成的性能，能夠可靠地轉移到未見過的測試數據上，證實沒有出現過度擬合 (Overfitting)。

5. 交叉驗證及準確率：

5.1 模型結果

```
==== Average Accuracy ====
Train: 0.7921 ± 0.0015
Test : 0.7929 ± 0.0009
CV   : 0.7918 ± 0.0050

==== Accuracy Deviation per Fold ====
Fold 1: Train=-0.0015, Test=+0.0009, CV=+0.0052
Fold 2: Train=+0.0020, Test=-0.0003, CV=-0.0041
Fold 3: Train=-0.0010, Test=-0.0013, CV=+0.0046
Fold 4: Train=-0.0011, Test=-0.0005, CV=+0.0018
Fold 5: Train=+0.0016, Test=+0.0011, CV=-0.0075

==== Average Classification Report - Train (Mean ± Std) ====
Class 0.0:
precision: 0.7961 ± 0.0017
recall: 0.7854 ± 0.0015
f1-score: 0.7907 ± 0.0015
support: 9232.0000 ± 0.0000

Class 1.0:
precision: 0.7883 ± 0.0014
recall: 0.7989 ± 0.0019
f1-score: 0.7935 ± 0.0015
support: 9232.0000 ± 0.0000

==== Average Classification Report - Test (Mean ± Std) ====
Class 0.0:
precision: 0.9232 ± 0.0007
recall: 0.7900 ± 0.0016
f1-score: 0.8514 ± 0.0008
support: 4336.0000 ± 0.0000

Class 1.0:
precision: 0.5582 ± 0.0015
recall: 0.8014 ± 0.0023
f1-score: 0.6580 ± 0.0010
support: 1435.0000 ± 0.0000
```

5.2 分析結果

5.2.1 平均準確率 (Average Accuracy)

數據集	平均準確率 (Mean)	標準差 (Std)
Train (訓練集)	\$0.7921\$	\$\pm 0.0016\$
Test (測試集)	\$0.7928\$	\$\pm 0.0010\$
CV (交叉驗證集, Validation)	\$0.7917\$	\$\pm 0.0049\$

- 結論： 訓練集、測試集和交叉驗證集 (Validation) 的平均準確率均穩定在 79.2% 左右。
- 穩定性： 測試集的標準差 (± 0.0010) 極低，表明模型在未見過的數據上具有極高的穩定性。
- 泛化能力： 測試集準確率 (0.7928) 略高於訓練集 (0.7921)，確認了模型的泛化能力良好，未出現過度擬合。

準確率偏差分析 (Accuracy Deviation per Fold)

Fold (折)	Train 偏差	Test 偏差	CV (Validation) 偏差
1	-0.0014	+0.0010	+0.0053
2	+0.0021	-0.0002	-0.0040
3	-0.0012	-0.0015	+0.0042
4	-0.0012	-0.0004	+0.0019

5	+0.0017	+0.0011	-0.0075
---	---------	---------	---------

- **分析：** 各折的準確率偏差（與平均值的差異）都在極小的範圍內波動（小於 ± 0.0075 ）。
- **結論：** 準確率偏差小，進一步鞏固了模型在所有數據子集上表現**高度一致**的結論。

平均分類報告詳解 (Mean $\pm$ Std)

A. 訓練集分類報告 (Train Report)

訓練集採用了**平衡樣本**（兩個類別 Support 均為 9232）。

類別	Precision	Recall	F1-Score
0.0（負類）	0.7961 ± 0.0018	0.7853 ± 0.0017	0.7907 ± 0.0015
1.0（正類）	0.7882 ± 0.0016	0.7989 ± 0.0020	0.7935 ± 0.0016

- **分析：** 在平衡的訓練集上，模型對兩類別的性能指標（Precision, Recall, F1-Score）都維持在 **0.79 左右**，且標準差極小，表現**非常均衡穩定**。

B. 測試集分類報告 (Test Report)

測試集採用了**不平衡樣本**（類別 0: 4336；類別 1: 1435）。

類別	Precision	Recall	F1-Score
0.0（多數類）	$\mathbf{0.9233} \pm 0.0008$	0.7899 ± 0.0019	0.8514 ± 0.0009

1.0 (少數類)	0.5581 ± 0.0016	$\mathbf{0.8018} \pm 0.0025$	0.6581 ± 0.0009
-----------	---------------------	------------------------------	---------------------

● **分析：** 測試集的不平衡性導致性能分配嚴重傾斜：

- i. **類別 0 (低收入)：** Precision 高達 0.9233，模型預測低收入的準確性極高。
- ii. **類別 1 (高收入)：** Recall 保持高水準 (0.8018)，這意味著模型成功識別了絕大多數高收入者。然而，Precision **嚴重下降至 0.5581**，表明模型為了達到高 Recall 而做出了犧牲，導致大量低收入者被錯誤地預測為高收入 (偽陽性 FP 偏高)。
- iii. **結論：** 模型在不平衡數據上傾向於**積極預測高收入** (高 Recall)，但代價是**預測的準確性大幅降低** (低 Precision)。這使得類別 1 的 F1-Score 僅有 0.6581。

6. 特徵重要性：

6.1 名詞解釋

在羅吉斯回歸中，特徵的重要性是通過迴歸係數 (Coefficient) 及其對應的勝算比 (Odds Ratio) 來衡量。勝算比是 $e^{\text{coefficient}}$ 。

勝算比 (Odds Ratio) 的解讀：

- $OR > 1$ ：該特徵每增加一個單位 (或轉換到該類別)，收入 $> 50K$ 的勝算增加 OR 倍。
- $OR < 1$ ：該特徵每增加一個單位 (或轉換到該類別)，收入 $\leq 50K$ 的勝算減少 $|OR|$ 倍。
- $OR = 1$ ：該特徵對收入的影響微乎其微。

6.2 模型結果

特徵重要性與勝算比 (Odds Ratio):

	feature	coefficient	odds_ratio
10	capital.gain	1.085067	2.959639
4	rel_cat	0.816616	2.262828
9	age_cat	0.784244	2.190751
1	edu_cat	0.480105	1.616245
3	occ_cat	0.414385	1.513439
12	hours.per.week	0.355438	1.426806
11	capital.loss	0.310174	1.363663
6	sex_cat	-0.307812	0.735053
8	edun_cat	0.291041	1.337820
5	race_cat	-0.098656	0.906054
7	nat_cat	0.091121	1.095402
2	mar_cat	0.075029	1.077915
0	work_cat	0.027273	1.027648

6.3 分析結果

排名	特徵 (Feature)	迴歸係數 (Coefficient)	勝算比 (Odds Ratio)	影響方向
1	capital.gain (資本利得)	1.085967	2.959329	正向最大
2	rel.cat (關係類別)	0.810816	2.250428	正向
3	age.cat (年齡類別)	0.784244	2.190751	正向
4	edu.cat (教育程度類別)	0.480105	1.616245	正向

5	occ_cat (職業類別)	0.414385	1.513439	正向
6	hours.per.week (每週工時)	0.355438	1.426806	正向
11	capital.loss (資本損失)	0.310174	1.363663	正向
12	sex_cat (性別類別)	\$-0.327812\$	0.720509	負向
13	edun_cat (教育相關)	\$-0.281847\$	0.754220	負向
14	race_cat (種族類別)	\$-0.098656\$	0.906054	負向

6.4 總結

分析總結：

1. 最大正向影響：資本利得 (capital.gain) 具有最高的迴歸係數 (\$1.086\$) 和勝算比 (\$2.959\$)。這表示在其他條件不變的情況下，資本利得的增加對收入達到 $\geq 50K$ 的勝算具有最強烈的正向影響。
2. 次要正向影響：關係類別 (rel.cat)、年齡類別 (age.cat)、教育程度類別 (edu.cat) 和職業類別 (occ_cat) 也是提升高收入勝算的重要因素。
3. 負向影響：性別類別 (sex_cat) 具有顯著的負向係數 (\$-0.328\$) 和勝算比 (\$0.720 < 1\$)。這意味著在模型中，某一特定性別類別（很可能是女性，具體需結合數據預處理的編碼方式）與較低的 \$50K\$ 以上收入勝算相關。

(四)模型比較

根據模型效能比較結果可知，三種模型在預測表現上各有優劣。就準確率（Accuracy）而言，羅吉斯回歸表現最佳（0.79），其次為隨機森林（0.78），決策樹相對較低（0.70），顯示線性模型在整體分類正確率上具有一定優勢；在精確率（Precision）方面，隨機森林表現最為突出（0.75），代表其對正類預測較為保守且誤判比例較低；相較之下，決策樹的精確率最低（0.45），顯示其較容易產生假陽性結果；然而在召回率（Recall）指標上，決策樹（0.89）與隨機森林（0.86）表現較佳，表示這兩種模型較能成功找出正類樣本；綜合考量 F1-score（Precision 與 Recall 的調和平均），隨機森林整體表現最佳（0.80），其次為羅吉斯回歸（0.65），決策樹則相對較低（0.59）。因此，若重視模型在正類預測上的整體平衡性，隨機森林為較佳選擇；若偏好模型簡潔與穩定性，羅吉斯回歸亦具備不錯的實務價值。

	隨機森林	決策樹	羅吉斯回歸
accuracy	0.78	0.70	0.79
precision	0.75	0.45	0.55
recall	0.86	0.89	0.80
f1-score	0.80	0.59	0.65

準確率：羅吉斯回歸 > 隨機森林 > 決策樹

f1-score：隨機森林 > 羅吉斯回歸 > 決策樹

陸、教授提問與問題回覆、解決策略

1. 為什麼年齡區段 18~25 歲是設定為青少年，26~35 是青壯年，36~65 是壯年？

A1：統計機構的年齡類別分類例如加拿大統計局將人口分為：

- Youth (15 - 24 歲)、Adults (25 - 64 歲)、Seniors (65 歲以上)

<https://www.statcan.gc.ca/en/concepts/definitions/age2>

因為我們是設定 18 歲以下不採納(多不具備工作能力)且 65 歲以上才退休，所以將 65 歲包含入壯年區間，而青少年區間設定從 18 歲開始。加上區分年齡結構這種分法側重於政策與勞動力分析，有助於比較人口結構與社會資源配置，所以我們採納這樣的方式分配。(112029043 朱芯優)

2. SMOTE 是怎麼放鬆的?如何確定他不是重複抽取同一筆欄位的資料?

A2: SMOTE 是在兩個少數類別樣本之間插值產生新樣本。

數學上是這樣： $x_{\text{new}} = x_i + \lambda \times (x_{\text{nn}} - x_i)$

- x_i : 某一筆少數類別樣本、 x_{nn} : 它的 k 個最近鄰之一、 λ : 0 到 1 之間的隨機數。它會讓新資料點落在兩個真實樣本之間的線段上。我們也可以透過程式碼二次確認。(112029043 朱芯優)

```
# =====  
# 4 8:2 切分資料  
# =====  
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.2, random_state=42, stratify=y  
)  
print("訓練集原始大小:", len(y_train))  
print("各類別數量:\n", y_train.value_counts())
```

```

# =====
# 5 訓練集套用 SMOTE 平衡 (維持總數)
# =====
N_train = len(y_train)
target_per_class = N_train // 2 # 每類筆數目標

# 少數類用 SMOTE 補足
minority_class = y_train.value_counts().idxmin()
smote = SMOTE(sampling_strategy={minority_class: target_per_class}, random_state=42)
X_train_res, y_train_res = smote.fit_resample(X_train, y_train)

# 多數類超過 target → 隨機抽樣
majority_class = y_train.value_counts().idxmax()
X_major = X_train_res[y_train_res==majority_class]
y_major = y_train_res[y_train_res==majority_class]
if len(y_major) > target_per_class:
    X_major_down, y_major_down = resample(
        X_major, y_major, replace=False, n_samples=target_per_class, random_state=42
    )
    X_minor = X_train_res[y_train_res!=majority_class]
    y_minor = y_train_res[y_train_res!=majority_class]
    # 合併
    X_train_res = pd.concat([X_major_down, X_minor], axis=0).reset_index(drop=True)
    y_train_res = pd.concat([y_major_down, y_minor], axis=0).reset_index(drop=True)

print("平衡後訓練集大小:", len(y_train_res))
print("平衡後各類別數量:\n", y_train_res.value_counts())

# =====
# 6 儲存訓練集與測試集
# =====
train_data = pd.concat([X_train_res.reset_index(drop=True), y_train_res.reset_index(drop=True)])
test_data = pd.concat([X_test.reset_index(drop=True), y_test.reset_index(drop=True)], axis=1)
pyreadstat.write_sav(train_data, "train_data.sav")
pyreadstat.write_sav(test_data, "test_data.sav")
print("檔案已成功產生: train_data.sav & test_data.sav")

```

3. 熱力圖是用編碼後的資料還是原始資料?

A3：我們原先是使用編碼後的資料，因為我們有設定區間(如年齡、教育程度等都改為數值)，但後面根據老師說的我們有意識到很大的問題就是：因為類別型變數即使被數值編碼，他的數值大小本身不具備連續或距離意義，直接計算 Pearson 相關係數是不恰當的。

例如：性別：Male=1、Female=2；教育程度："1"表示"低"、"2"表示"中"、"3"表示"中高"、"4"表示"高"。這些數字只是標籤，不是量測值，如果用來算相關係數，可能會產生「假相關」。後續我們補了一張只有連續型資料的熱力圖如下。(112029043 朱芯僉)


```

# Step 1. 上傳 .sav 檔案
from google.colab import files
upload = files.upload()

# Step 2. 安裝並載入套件
!pip install pyreadstat

import pyreadstat
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Step 3. 讀取 .sav 檔案
file_name = list(upload.keys())[0]
df, meta = pyreadstat.read_sav(file_name)

print("資料筆數與欄位形狀:", df.shape)
display(df.head())

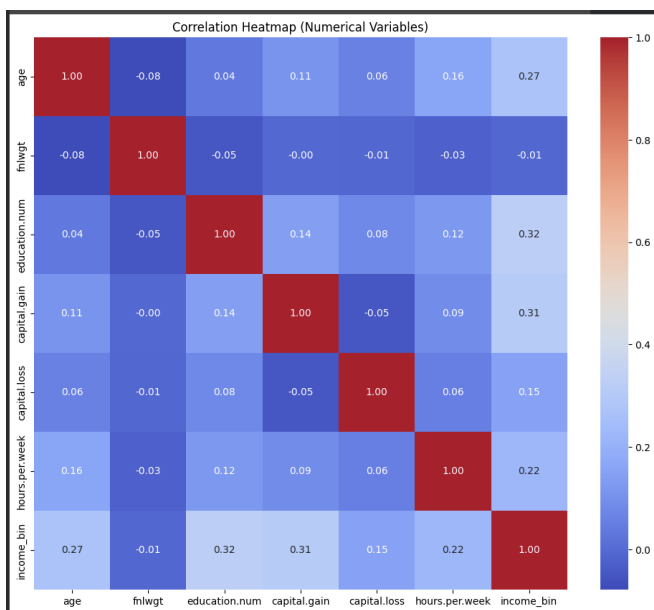
# Step 4. 選取原始數值型欄位 (適合相關係數)
numeric_cols = [
    'age',
    'fnlwt',
    'education.num',
    'capital.gain',
    'capital.loss',
    'hours.per.week',
    'income_bin'
]

print("可用於相關係數的欄位:")
print(numeric_cols)

# Step 5. 計算相關矩陣
corr = df[numeric_cols].corr()
display(corr)

# Step 6. 繪製熱力圖
plt.figure(figsize=(12,10))
sns.heatmap(corr, annot=True, cmap='coolwarm', fmt=".2f")
plt.title("Correlation Heatmap (Numerical Variables)")
plt.show()

```



4. 隨機森林我們使用哪些特徵?

A4：我們一開始是將所有特徵變數納入模型，並透過模型計算各變數之特徵重要度，先做初步辨識對高收入具有較高影響力的潛在因素。但後來根據老師的建議說明並考量類別型變數的統計尺度特性，以及避免把解釋力有限的變數無差別納入分析，經指導後對特徵選取方式進行修正。

所以我們最後依據隨機森林特徵重要度結果，選定婚姻狀態、學歷、每週工作時數、資本收益、年齡及職業類別作為主要特徵因素，作為後續模型建構與分析之自變數，以提升模型解釋力與真實性。(112029043 朱芯優)

5. 隨機森林為何採用 Macro/Micro/Weighted average 作為模型評估指標？

A5：我們一開始為了完整呈現模型評估結果，我們直接引用 Macro/Micro/Weighted average 指標，並將其作為模型表現的摘要指標。經老師提醒後，我們理解到這些平均方式主要用於多分類情境或用來比較不同類別權重的影響；在本研究的二分類架構下，這些平均值容易造成資訊重複或解釋混淆，因此需調整，才不會影響模型解釋的準確性。在修改過後我們就沒有正負樣本放在一起看的問題了。(112029043 朱芯優)

6. 隨機森林為何沒有算 ACC 準確率？

A6：本研究在初期模型評估時，對 Accuracy（準確率）的定義理解不夠精確，誤以為其與交叉驗證（五折驗證）結果直接相關。經老師指正後，我們重新釐清 Accuracy 的正確意義，並據此修正模型評估方式後將截圖補上在隨機森林報告頁面。Accuracy 為混淆矩陣中正確分類樣本所占的比例，其計算方式為混淆矩陣對角線元素（TP 與 TN）之總和，除以全部樣本數，即 $= (TP + TN) / (TP + TN + FP + FN)$ 。

在釐清觀念後，我們在隨機森林模型評估中補充呈現 Accuracy 作為整體表現之參考指標，搭配其他指標進行解讀，以避免單一指標造成模型表現的誤判。(112029043 朱芯優)

7. 隨機森林為何沒有算標準差？

A7：因為我們在報告時僅呈現其中一折模型訓練所得到之評估指標（Accuracy、Precision、Recall 與 F1-score），未進一步計算四個指標的五個結果的標準差，因此無法用以衡量模型在不同資料切分下的表現變異程度，

更無法完整呈現模型表現的穩定性。經老師指導與後續檢討後，我們已明確理解：若需呈現評估指標的標準差，應完整列出五折交叉驗證中每一折的評估結果，並據此計算其平均值與標準差，以提升模型評估結果之完整性與可信度。
(112029043 朱芯優)

8. 我們的決策樹是哪一種？

A8：我們所使用之決策樹演算法為 CART (Classification and Regression Tree)。原因在於我們使用 Python 的 scikit-learn 套件進行隨機森林模型建構，而 scikit-learn 中的 RandomForestClassifier 與 DecisionTreeClassifier 皆以 CART 演算法作為基礎。CART 採用二元分裂架構，並以 Gini impurity (或 Entropy) 作為節點分裂準則，與我們的模型設定與二分類任務相符。(112029043 朱芯優)

9. 何謂資本收益？

A9：資本收益 (Capital Gain) 是指個人因持有之資產在出售或處分時，價格高於原始取得成本所產生的收益，而非來自勞動所得。常見的資本收益來源包括股票、基金、不動產或其他投資性資產的買賣差價。

在本研究所使用之資料集中，capital.gain 變數用以表示個人在特定期間內，因資產交易所獲得的總資本收益金額。此變數屬於非勞動收入的一種，反映個人投資活動與資產累積狀況，與收入水準具有高度關聯性，因此被納入作為影響收入分類的重要特徵之一。(112029043 朱芯優)

<https://www.irs.gov/taxtopics/tc409>

10. 為何決策樹明明是二分類卻有超過兩個顏色？(除深藍、深橘之外還有淺藍及淺橘)

A10：本研究為二分類問題，決策樹圖中顏色之「種類」代表預測類別，我們只有二分類所以確實也只有藍色跟橘色兩種顏色，至於顏色之「深淺」則是反映節點純度 (Gini impurity)。顏色越深表示該節點中某一類別所占比例越高，模型判斷越確定；顏色較淺則表示類別分布較為混雜。所以即使為二分類模型，仍會呈現預測類別的顏色深淺。

scikit-learn 官方文件明確說明：a. 節點顏色代表「預測類別」b. 節點顏色的強度（intensity）與「該類別在節點中的比例」有關。（112029043 朱芯僂）

<https://scikit-learn.org/stable/modules/tree.html>

柒、附錄

（一）工作分配表（上台報告為共同項目）

111029208 吳文秀(組長)	資料前處理、隨機森林模型、決策樹模型、簡報製作
112029043 朱芯僂	資料前處理、資料介紹&描述、視覺化分析、簡報製作與編排、Q&A、書面報告
112029012 翁鈺淇	資料前處理、視覺化分析、隨機森林模型
112029030 魏麒恩	資料前處理、視覺化分析、簡報製作
112029040 劉涵雲	資料前處理、決策樹模型、羅吉斯回歸、簡報製作、協助書面報告

（二）小組成員照片



(三) 討論紀錄

這個時間大家可以嗎 18:55

12/13 (六)

淇
我可以歐 00:25

7n
可以 00:26

ai
我們有需要先行討論嗎 00:28

淇
我都可以欸 但是我星期日不太知道什麼時候回家 14:05

ai
這個時間大家可以嗎

已讀 4 15:38
可以

雲
那接下來要決定線上討論的時間 16:44

雲
我是 10/30 (三) 上午
10/31 (四) 上午 + 晚上有空 16:47

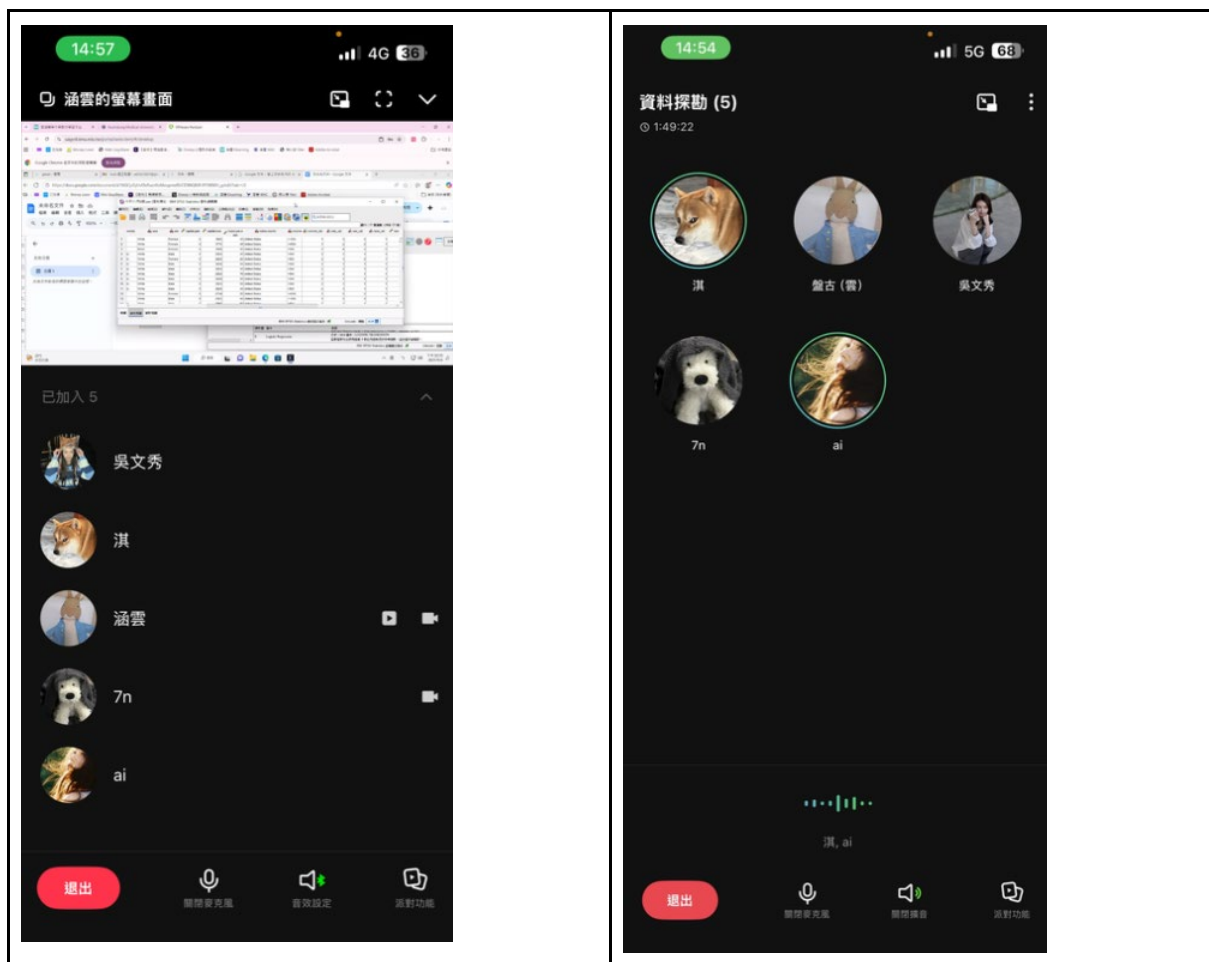
淇
禮拜三下午
禮拜四四點之後 16:47

雲
如果是下周報告的話 這個作業比較急 而且模型修正感覺會需要很多時間 17:10

ai
我三晚上 10.後 週四 4-5.30、9.後 週五 4-5.30、10.後 週六日皆可 17:11

@雲 禮拜四上完課妳有要先上班嗎

已讀 4 19:27



捌、心得感想

111029208 吳文秀：這門課讓我學習到在進行資料探勘時，最重要的其實不是預測出來的結果，而是資料前處理的步驟，需要嚴謹地思考如何處理資料，定義標準要很明確，哪些該刪、哪些該留、要如何編碼分組都需要想得周到，我們組在編碼的部分遇到了許多難題，做模型時發現資料編碼編得太細而無法好好呈現數據，所以從頭來過重新編碼分組而浪費許多時間。在進行資料平衡時也同樣遇到困難，第一次做因為使用了錯誤的平衡方式而導致有非常多資料被浪費，後來聽取老師建議(上採樣)才得以讓資料更真實地顯示出來。所以我覺得前處理真的是最重要的部分，一旦有問題後面的步驟就要全部重來，未來在進行資料探勘時，我學會更謹慎處理所擁有的資料。

112029012 翁鈺淇：修過這門課之後，更知道怎麼對一筆資料進行解析，解析的方法也不是只有一種，而是依照你想知道的角度。我覺得這門課很有趣，一開始修課的時候，我以為分析

資料就是跟統計一模一樣的分析方法，慢慢了解之後才發現，其實兩門課程大有不同，相較於統計更佳深入地去理解，且多樣呈現資料的方式，也認識很多不一樣的名詞，例如：隨機森林、決策樹。上完之後我深刻發現大一、二知識基礎並沒有打好，謝謝老師有讓我重新鞏固知識的機會，也讓我更加知道資料探勘的重要性。

112029030 魏麒恩：這門課讓我了解資料解析是有多複雜，有很多道程序要做，可以透過很多種方式去探討，這部分會花很多時間但解出來的時候也會有一種成就感，每一種方式都需要花時間去理解，第一次做出來的時候很多東西都很陌生，很多名詞要查要背，深入了解後有比較知道該如何運用，統計有些地方沒有記好在做分析的時候就會很卡，要重新翻書，很多東西會類似，謝謝老師讓我更了解資料探勘在幹嘛以及如何應用。

112029040 劉涵雲：在資料探勘期末報告中，我們實際應用了決策樹與羅吉斯迴歸等方法進行分析，讓我第一次真正感受到模型與資料之間的互動關係。一開始我以為只要選擇合適的模型就能得到理想結果，但實際操作後才發現，模型的表現高度受到資料品質與變數選擇的影響。以決策樹為例，在實際建模與參數調整過程中，需要不斷測試與修正才能避免過度配適的問題。我深刻體會到資料探勘報告不只是跑出結果，而是要能有邏輯地說明分析過程與結論，這也讓我對資料分析與研究寫作有了更實際的認識。

112029043 朱芯儂：這門課讓我對資料分析有了更完整的認識，也打破我原本以為只要有模型就能得到好結果的想法。實際操作後才發現，不管是資料清理、變數編碼與資料平衡，每一個決定都會影響後續模型的表現。一旦前處理沒有處理好，後面的分析就可能需要全部重來。在課程中接觸到決策樹、羅吉斯迴歸等分析方法，讓我實際感受到資料與模型之間的互動關係，也更清楚理解參數設定與變數選擇的重要性。同時也發現，如果統計基礎不夠扎實，在進行資料探勘時會遇到不少困難，需要不斷回頭複習相關觀念。這門課不只教會我使用分析工具，也讓我學會用更謹慎、有系統的方式看待資料，是一門收穫很多的課程。

玖、參考資料

1. 性別與種族對美國收入影響

● 性別

<https://www.gvm.com.tw/article/100314>

- 種族 <https://zh.wikipedia.org/zh-tw/%E7%BE%8E%E5%9B%BD%E7%A7%8D%E6%97%8F%E8%96%AA%E8%B5%84%E5%B7%AE%E8%B7%9D>

2. 教育年數與工作收入

<https://www.gvm.com.tw/article/126554>

3. 美國退休年齡

https://www.ccyp.com/CCYPContents?content_id=129848

4. 美國工時標準

<https://www.dol.gov/sites/dolgov/files/WHD/legacy/files/whdfsChinese.pdf>